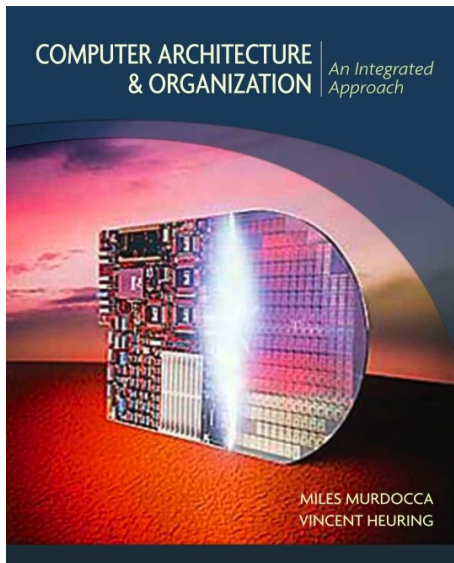


Organización de las Computadoras

Leonardo Giovanini



Organización de la memoria

Contenidos

6.1 Desarrollo y ejecución de programas

6.2 Jerarquía de almacenamiento

6.3 Generación de las direcciones

6.4 Manejo de memoria

6.4.1 *Paginación*

6.4.2 *Segmentación*

6.4.3 Virtualización

6.5 Unidad de manejo de memoria

Desarrollo y ejecución de programas

Desarrollo y ejecución de un programa

Compilación

Un **Compilador** traduce un programa escrito en **lenguaje de alto nivel** (código fuente), el cual es *independiente de la arquitectura* de la CPU, en **lenguaje ensamblador**, el cual *depende de la arquitectura* de CPU utilizada.

Código ensamblador

```
.data
f:
g:
y:

.text
main:
    addi $sp, $sp, -4 # make stack frame
    sw   $ra, 0($sp) # store $ra on stack
    addi $a0, $0, 2   # $a0 = 2
    sw   $a0, f       # f = 2
    addi $a1, $0, 3   # $a1 = 3
    sw   $a1, g       # g = 3
    jal  sum          # call sum procedure
    sw   $v0, y        # y = sum (f, g)
    lw   $ra, 0($sp)  # restore $ra from stack
    addi $sp, $sp, 4   # restore stack pointer
    jr   $ra          # return to operating system

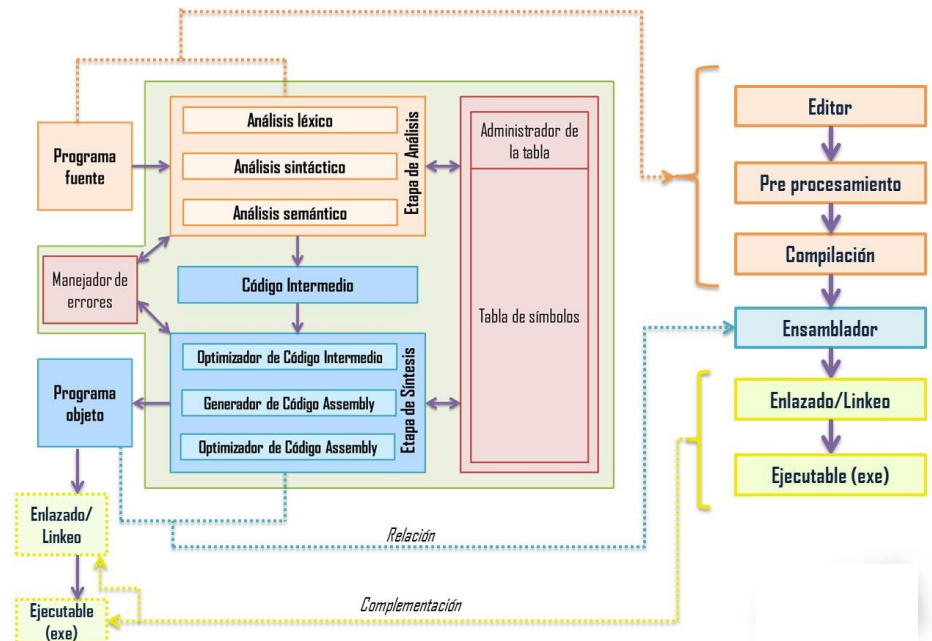
sum:
    add  $v0, $a0, $a1 # $v0 = a + b
    jr   $ra          # return to caller
```

Código fuente

```
int f, g, y; // global variables

int main (void)
{
    f = 2;
    g = 3;
    y = sum (f, g);
    return y;
}

int sum (int a, int b) {
    return (a + b);
}
```



Desarrollo y ejecución de un programa

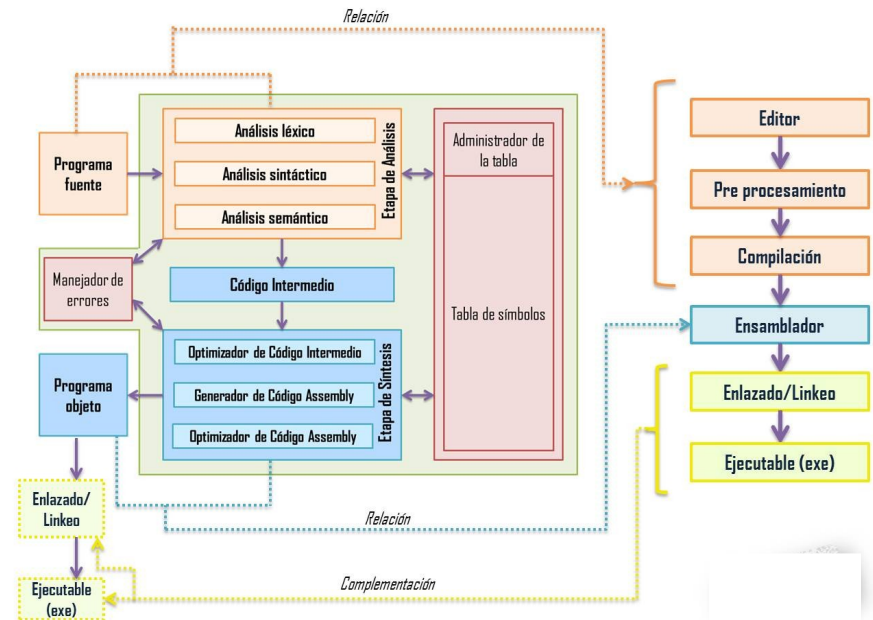
Ensamblado

El **lenguaje ensamblador** es un lenguaje de programación de bajo nivel que consiste en un conjunto de mnemónicos que representan instrucciones básicas de los procesadores.

Implementa una **representación simbólica** de los códigos de máquina binarios e información necesarias para programar un procesador y es la representación **directa del código máquina**.

```

0x00400000  main:  addi $sp, $sp, -4
0x00400004          sw  $ra, 0($sp)
0x00400008          addi $a0, $0, 2
0x0040000C          sw  $a0, f
0x00400010          addi $a1, $0, 3
0x00400014          sw  $a1, g
0x00400018          jal  sum
0x0040001C          sw  $v0, y
0x00400020          lw  $ra, 0($sp)
0x00400024          addi $sp, $sp, 4
0x00400028          jr   $ra
0x0040002C  sum:  add  $v0, $a0, $a1
0x00400030          jr   $ra
  
```



Un **ensamblador** traduce programa en **lenguaje ensamblador** en **código de máquina** en dos pasos:

- Asigna una dirección a cada instrucción y busca los símbolos (etiquetas y nombre de variables);
- Produce el código de máquina y la tabla de símbolos.

El código de máquina y la tabla de símbolos son almacenadas en el **archivo objeto**.

Desarrollo y ejecución de un programa

Enlazado

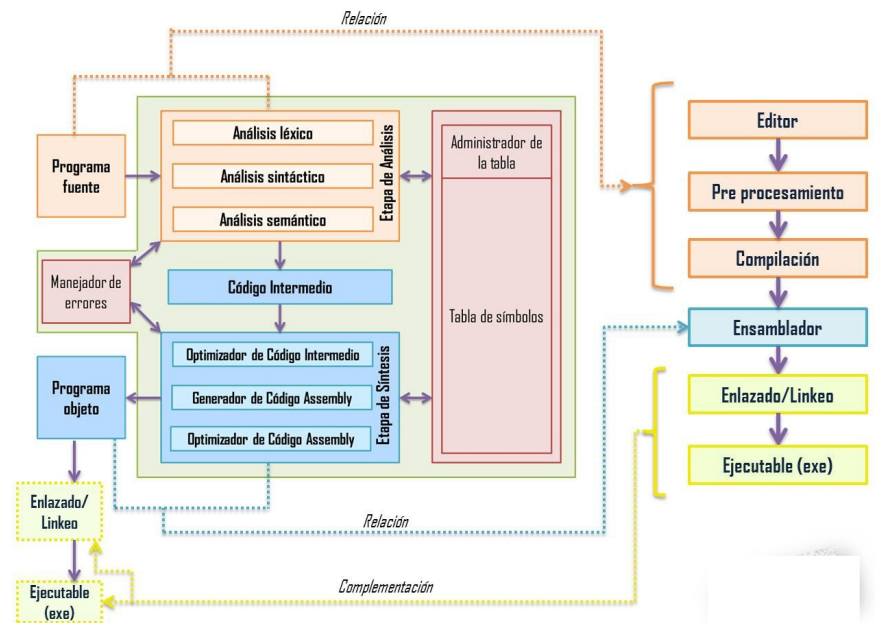
El **enlazador** organiza los datos e instrucciones en un archivo que puede **ser ejecutado** por el procesador una vez cargado en memoria.

Text Size	Data Size
0x34 (52 bytes)	0xC (12 bytes)
Address	Instruction
0x00400000	0x23BDDFFC
0x00400004	0xAFBF0000
0x00400008	0x20040002
0x0040000C	0xAF848000
0x00400010	0x20050003
0x00400014	0xAF858004
0x00400018	0x0C10000B
0x0040001C	0xAF828008
0x00400020	0x8FBF0000
0x00400024	0x23BD0004
0x00400028	0x03E00008
0x0040002C	0x00851020
0x00400030	0x03E00008
Address	Data
0x10000000	f
0x10000004	g
0x10000008	y

```

addi $sp, $sp, -4
sw  $ra, 0 ($sp)
addi $a0, $0, 2
sw  $a0, 0x8000 ($gp)
addi $a1, $0, 3
sw  $a1, 0x8004 ($gp)
jal  0x0040002C
sw  $v0, 0x8008 ($gp)
lw  $ra, 0 ($sp)
addi $sp, $sp, -4
jr  $ra
add $v0, $a0, $a1
jr  $ra

```

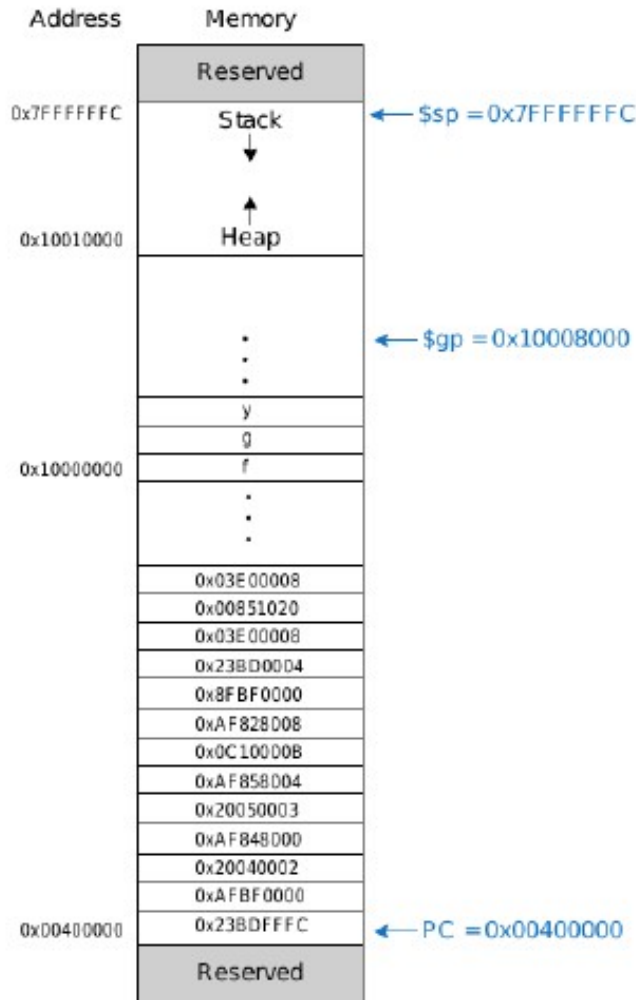


Un enlazador toma los objetos generados en el proceso de compilación, la información de todos los recursos necesarios (biblioteca), quita aquellos recursos que no necesita, y enlaza el código objeto con sus bibliotecas con lo que finalmente produce un fichero ejecutable.

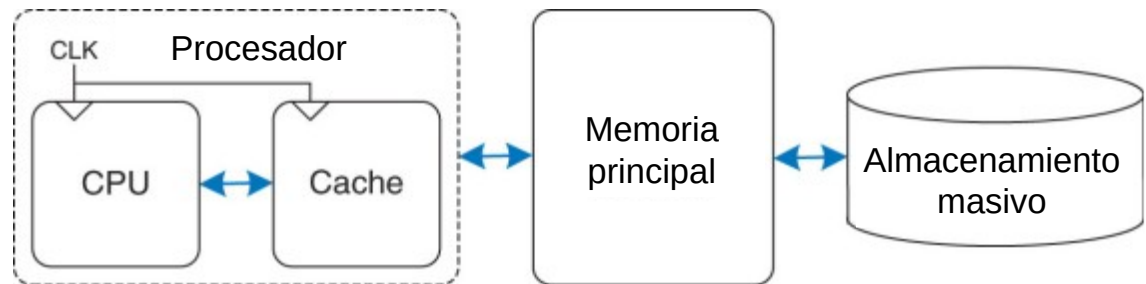
En el caso de los programas enlazados dinámicamente, el enlace entre el programa ejecutable y las bibliotecas se realiza en tiempo de carga o ejecución del programa.

Desarrollo y ejecución de un programa

Carga y Ejecucion



El sistema operativo lee el archivo ejecutable del dispositivo de almacenamiento y lo carga en la memoria de la computadora para su ejecución.



Dependiendo del entorno utilizado,

- En **procesadores avanzados** debe ubicarse en el contexto de un **proceso**, y la gestión esta a cargo de la unidad de manejo de memoria,
- En **procesadores convencionales** se ubica en la memoria disponible y la gestión esta a cargo del programador.

La ubicación de cada segmento del programa (código, datos, variables locales y pila) depende de las características técnicas del procesador definidas en el ISA.

Desarrollo y ejecución de un programa

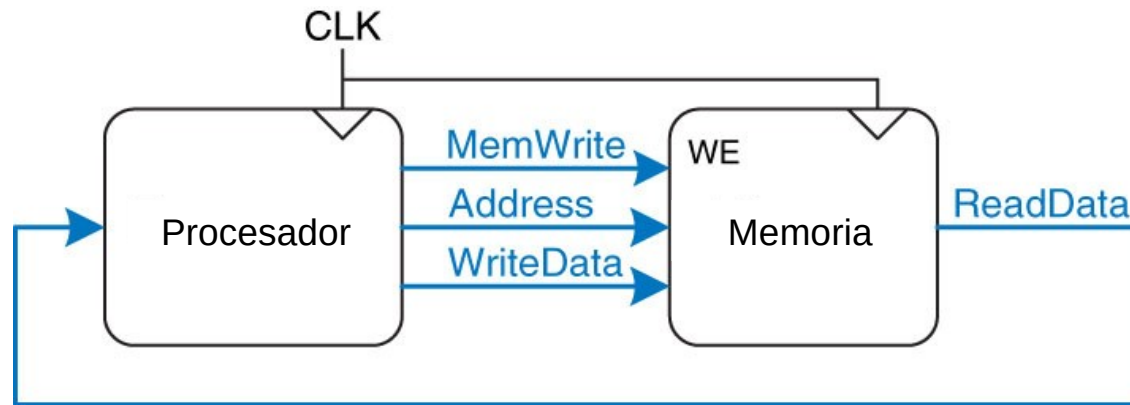
Ejecucion

Los **registros y la memoria son los únicas formas de almacenamiento** a la que el procesador puede acceder directamente para **ejecutar un programa**.

Sin embargo, puede almacenar datos y programas en perifericos.

La unidad de memoria sólo ve desde el procesador:

- ✓ Requerimientos de lectura mas las correspondientes direcciones;
- ✓ Requerimientos de escritura mas las correspondientes direcciones y datos.

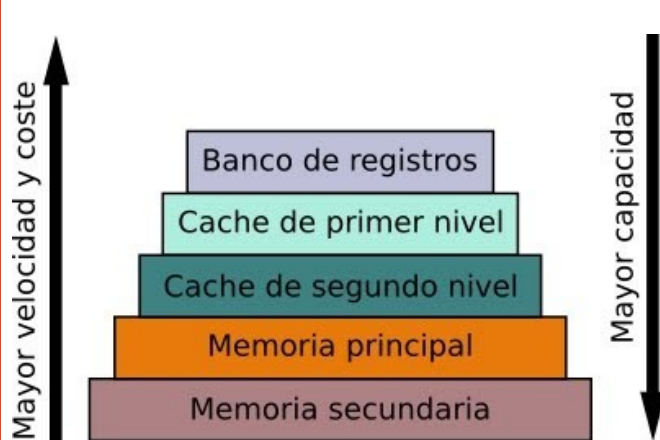


Por lo tanto, el **desempeño de una computadora** depende de los **desempeños de su procesador y su sistema de memoria**

$$\text{Instruction time} = (\text{CPU clock cycles} + \text{Memory-stall clock cycles}) * \text{Cycle time}$$

Jerarquía de almacenamiento

Jerarquía de almacenamiento



Para mejorar el desempeño de una computadora y mantener su costo bajo el sistema de almacenamiento esta organizado en una jerarquía determinada por

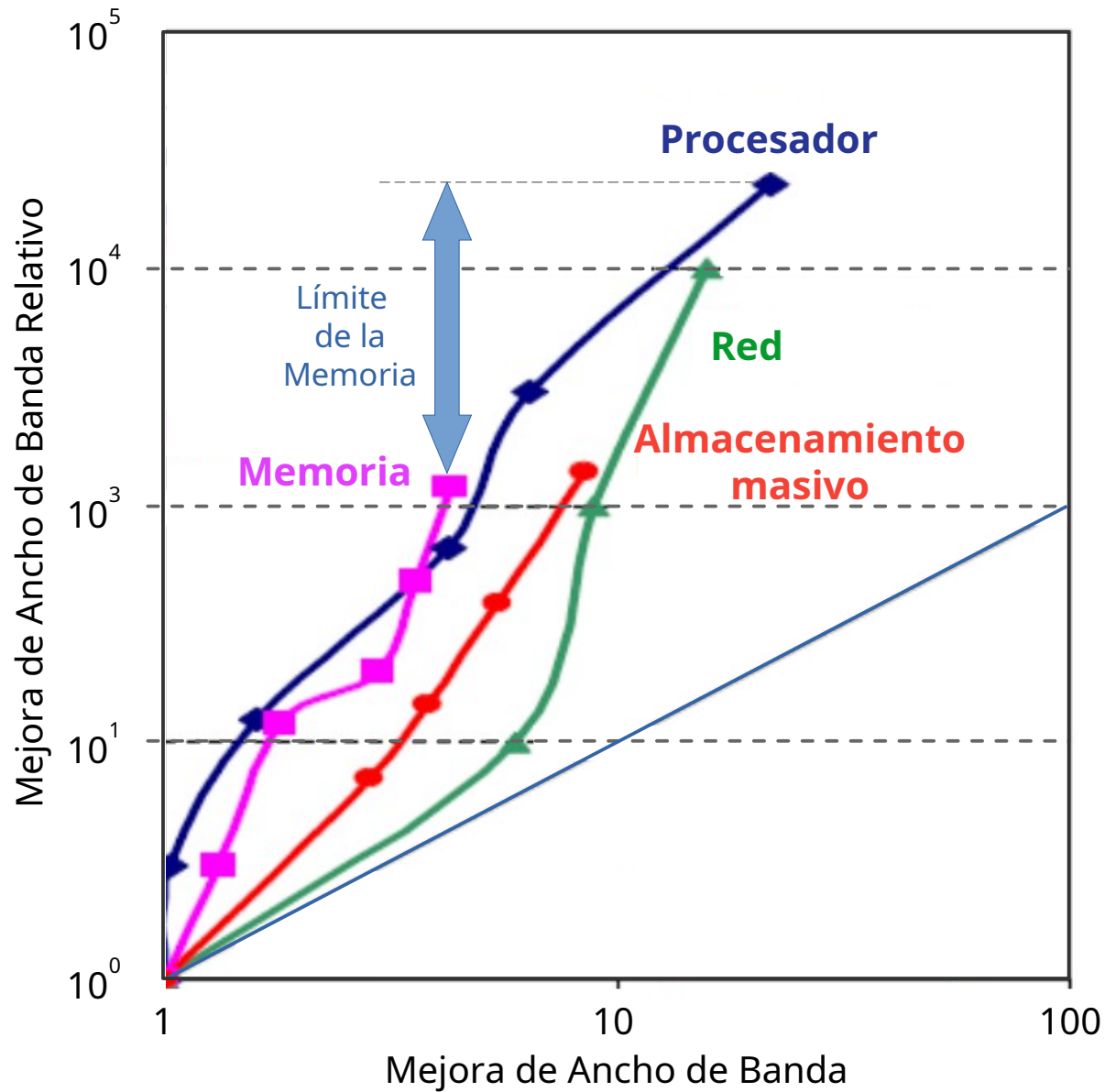
- **Velocidad**
- **Costo**
- **Volatilidad**

Hay cuatro niveles de almacenamiento

- **Interno** – Esta constituido por los registros del procesador y consiste en una pequeña cantidad de almacenamiento rápido, aunque algunos tienen funciones de hardware específicas. Este nivel se trata con mecanismos de direccionamiento distintos a la memoria principal; **CACHE**
- **Principal** – Esta constituida por la memoria del sistema y las memorias caches. Consiste en circuitos integrados computadora que almacenan información para uso inmediato. Esta organizada en palabras de longitud de palabra fija que agrupan celdas de memoria que almacena un bit; **RAM**
- **Almacenamiento masivo on-line (secundario)** – Esta constituido por las unidades de disco duro, estado sólido y discos flash USB. No es accesible directamente por el procesador, el cual utiliza canales de entrada/salida para accederlos y transferir los datos deseados al almacenamiento primario. El almacenamiento masivo on-line retiene datos cuando se apaga la alimentación. **HDD / SDD**
- **Almacenamiento masivo off-line (terciario)** – Esta constituida por unidades de almacenamiento masivos extraibles (cintas magneticas, discos ópticos). Se utiliza para archivar información a la que rara vez se accede, de modo que se utiliza para almacenar y/o transportar cantidades extraordinariamente grandes de datos. **CD / DVD**

Desarrollo y ejecución de un programa

Ejecución



Jerarquia de almacenamiento



Level	1	2	3	4	5
Name	registers	cache	main memory	solid state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25 - 0.5	0.5 - 25	80 - 250	25,000 - 50,000	5,000,000
Bandwidth (MB/sec)	20,000 - 100,000	5,000 - 10,000	1,000 - 5,000	500	20 - 150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape



Jerarquía de almacenamiento

Escala de tiempos

Evento	Latencia	Escala
Ciclo de CPU	0,3 ns	1 s
Acceso a memoria Cache Nivel 1	0,9 ns	3 s
Acceso a memoria Cache Nivel 2	2,8 ns	9 s
Acceso a memoria Cache Nivel 3	12,9 ns	43 s
Acceso a memoria Principal	120 ns	6 min
Acceso a disco estado sólido (SDD)	50 – 150 μ S	2 – 6 días
Acceso a a disco rígido mecánico (HDD)	1 – 10 ms	1 – 12 meses
Acceso a internet	40 – 183 ms	4 – 19 años
Retransmisión de paquete TCP	1 – 3 s	105 – 317 años
Reinicialización desde sistema operativo (software)	4s	423 años
Reinicialización desde sistema operativo (hardware)	40s	4000 años
Reinicialización física	5 min	32000 años

Generación de las direcciones

Generación de las direcciones

Durante su desarrollo, un programa pasa a través de diferentes pasos (**Compilación**, **Enlazado** y **Carga**) antes de ser ejecutado.

Las direcciones son representadas de diferentes maneras a lo largo de estas etapas.

- El código fuente típicamente utiliza direcciones simbólicas;
- Un compilador típicamente relaciona la dirección simbólica con una dirección relocalizable; y
- El enlazador y/o el cargador relacionan la dirección relocalizable con una dirección absoluta (física).

La relación (**binding**) entre las direcciones de las instrucciones y datos con las direcciones en memoria se pueden calcular durante:

- **La compilación** - si uno conoce la posición de memoria donde se ubicará el proceso;
- **La carga** - si no se conoce la dirección de memoria. En este caso, el compilador debe generar un **código relocalizable** cuya dirección de inicio cambia cada vez que se carga; y
- **La ejecución** - si el proceso puede moverse durante la ejecución desde un lugar de memoria a otro.

Generación de las direcciones

Direcciones lógicas y físicas

Una dirección **identifica de forma única** una posición en la memoria.

Hay dos tipos de direcciones: la **dirección lógica** y la **dirección física**.

La dirección lógica es la **dirección vista por el usuario** (programa) y se utiliza como una **referencia** para acceder a la dirección física.

La dirección física que refiere a **una posición** en la unidad de memoria del sistema.

La **diferencia fundamental** entre la dirección lógica y la física es que el procesador genera una **dirección lógica durante la ejecución de un programa**, mientras que la **dirección física se genera cuando se accede la unidad de memoria**.

El **espacio de direcciones virtuales** es el conjunto de todas las direcciones a las que puede hacer referencia un programa, mientras que el **espacio de direcciones físicas** es el conjunto de todas las direcciones de memoria físicas a las que accede un programa.

Por lo tanto

- Las direcciones lógicas y físicas son **iguales** para la generación de direcciones en la **compilación** y la **carga** del programa;
- Las direcciones lógicas y físicas son **diferentes** para la generación de direcciones en la **ejecución** del programa.

Generación de las direcciones

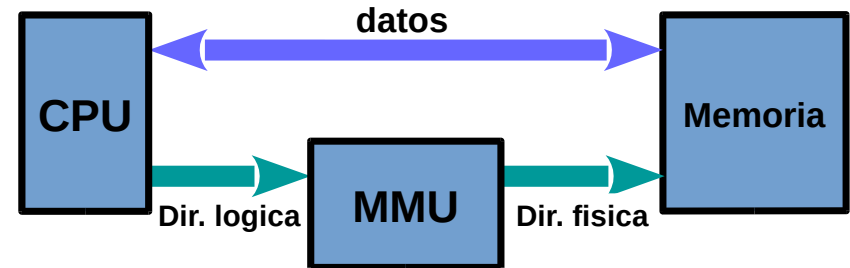
Direcciones logicas y fisicas

Los programas solo trabajan con direcciones lógicas

- Nunca ven las **direcciones físicas** utilizadas por el procesador durante su ejecución; y
- La dirección lógica es **independiente** de las direcciones físicas disponibles en el sistema.

Estas ideas obligan a introducir hardware especializado en el sistema: **la Unidad de Manejo de Memoria (MMU).**

Es un circuito especializado que (durante la ejecución) mapea las direcciones lógicas en direcciones físicas.



La dirección de salida del procesador a la unidad de administración de memoria es la dirección lógica mientras que la salida de la MMU es la dirección física, traduciendo las direcciones lógicas a direcciones físicas.

Las funciones de la MMU son

- **Gestionar** el acceso de los programas al sistema de memoria;
- **Automatizar** las transferencias de datos **entre niveles** en la jerarquía de memoria;
- **Aislar** la ejecución de los programas; y
- **Ampliar los espacios de direcciones** de las arquitecturas, conectando los procesadores a memorias físicas más grandes.

Manejo de memoria

Manejo de memoria

El manejo de memoria es un **conjunto de técnicas aplicados a la gestión de memoria** de una computadora para **asignar dinámicamente** la memoria necesaria a los programas que lo soliciten, y liberarla para reutilizarla cuando ya no sea necesaria.

El manejo de memoria realiza un **seguimiento del estado** de cada bloque de memoria, ya sea asignada o libre. Determina cómo se asigna la memoria entre los procesos, decidiendo cuánta memoria recibe, cuándo y donde se la asigna.

Las técnicas de manejo de memoria son

- **Paginación** – es una técnica que divide **la memoria física y el espacio de direcciones lógicas en unidades de tamaño fijo**. La MMU asigna unidades de memoria física con unidades de memoria lógica de modo que el espacio de direcciones lógicas es continuo. La paginación requiere soporte de hardware en la forma de tablas que contienen la dirección física y otros datos (protección de acceso y de estado, entre otros) de cada página;
- **Segmentación** - es una técnica de gestión que proporciona al usuario de un **espacio de direcciones lineal y contiguo**. Los segmentos son **áreas de memoria que corresponden a agrupaciones lógicas de información**. La segmentación requiere soporte de hardware en forma de tablas que contienen la dirección física, tamaño y otros datos (protección de acceso y de estado, entre otros) de cada segmento;
- **Memoria virtual** – es una técnica que separa las direcciones utilizadas por un proceso (direcciones lógicas) de las direcciones físicas reales, lo que permite la separación de procesos y aumenta el tamaño del espacio de direcciones lógicas más allá de la cantidad de memoria física disponible mediante el intercambio con almacenamiento secundario.

Manejo de memoria

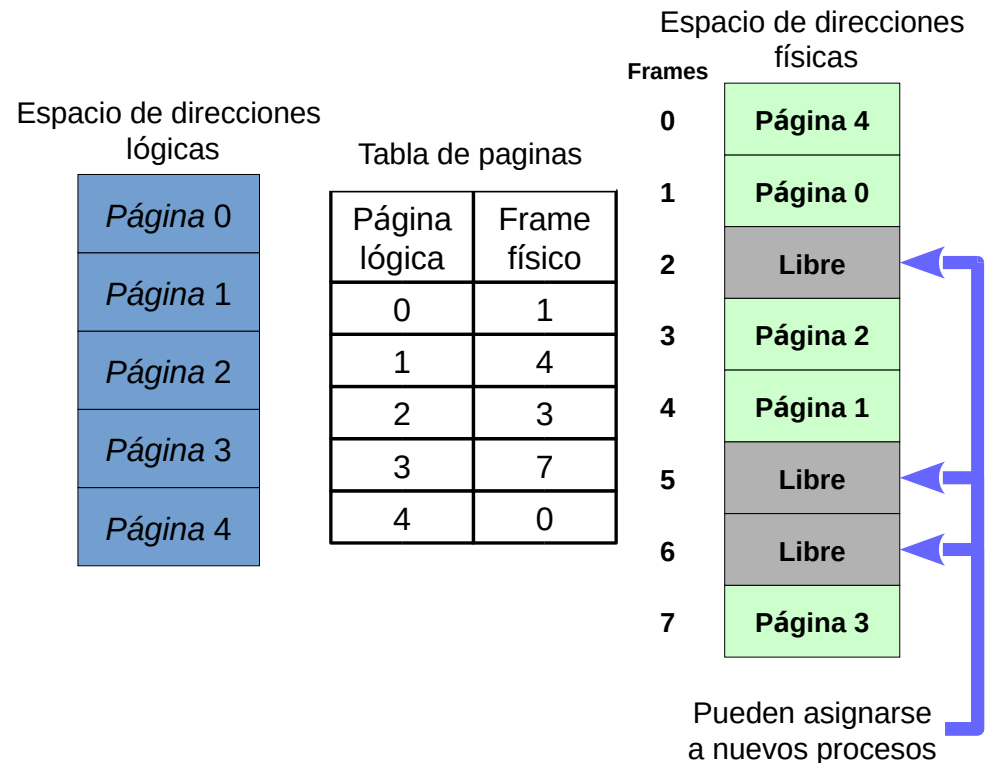
Paginación

La paginación es una técnica de gestión de memoria que divide la memoria física y el espacio de direcciones lógicas en unidades de tamaño fijo llamados frames y páginas, respectivamente.

A cada **página lógica** se le asigna a un **marco físico** de tamaño similar en la memoria principal.

Los marcos no tienen que ser contiguos en la memoria física, por lo que cada proceso puede dispersarse en toda la memoria y puede verse como una colección de cuadros que no son necesariamente contiguos.

Utiliza una **tabla de páginas** para realizar un seguimiento de dónde se encuentra cada página lógica dentro del espacio de las direcciones físicas y su estado.



Manejo de memoria

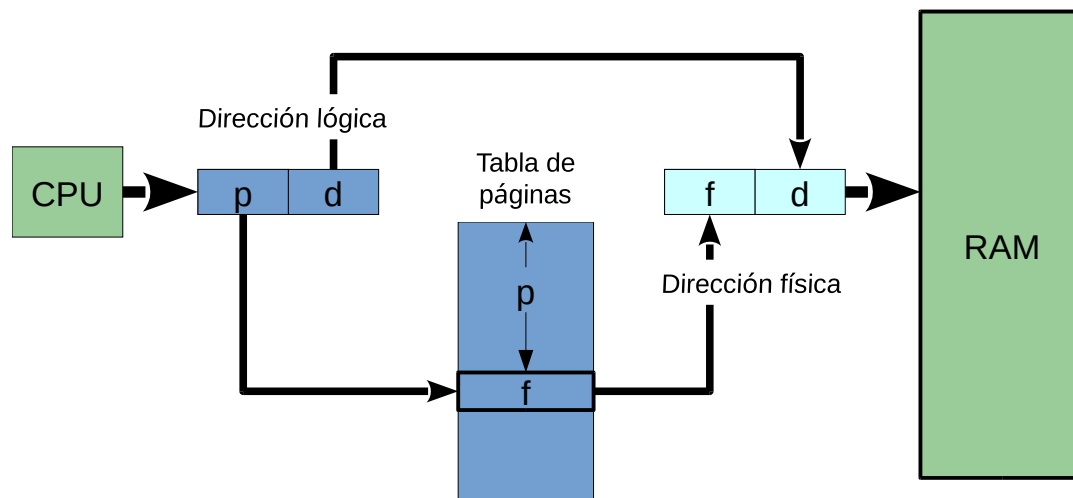
Paginación

La dirección lógica generada por el procesador es dividida en partes

- **Numero de pagina (p)** – utilizado como un índice en la **tabla de página** la cual contiene la dirección base de cada página en la memoria física; y
- **Offset de la pagina (d)** – combinado con la dirección base define la dirección física de la unidad de memoria.

Número de página	Desplazamiento de página
p	d

p	d
-----	-----



Esta información es almacenada en tablas que contienen las entradas de cada pagina y frame.

Una opción es usar un banco dedicado de registros de hardware para almacenar la tabla de páginas.

Las principales características de este enfoque son:

- Traducción rápida;
- Cambio de contexto lento - la tabla de páginas completa tiene que ser recargada; y
- Limitación de tamaño - algunas tablas de páginas pueden ser demasiado grandes.

Manejo de memoria

Paginación

Cada usuario tiene una tabla que contiene las entradas de cada pagina.

El espacio necesario por las tablas de paginas es proporcional al espacio de direcciones y el numero de procesos.

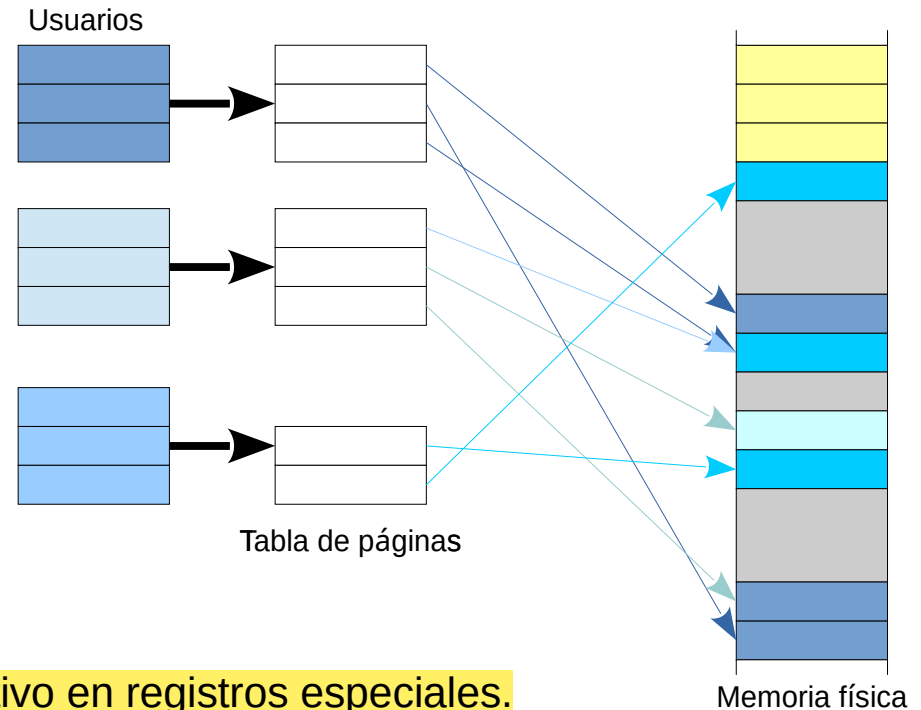
En general es muy grande para ser almacenadas en registros.

La idea es almacenar la tabla del proceso activo en registros especiales.

Las tablas de página se almacenan en la memoria principal y la información para accederlas se almacena en dos registros

- **Registro de base de la tabla de página (PTBR)** – la cual almacena la dirección física de la tabla de página de cada proceso; y
- **Registro de longitud de tabla de página (PTLR)** – la cual indica el tamaño de la tabla de página.

Cambiar la tabla de página sólo requiere cambiar dos registros.



Paginación

La solución es utilizar una tabla de hardware de rápida lectura basada en memorias asociativas llamada **Translation Look-aside Buffer (TLB)**.

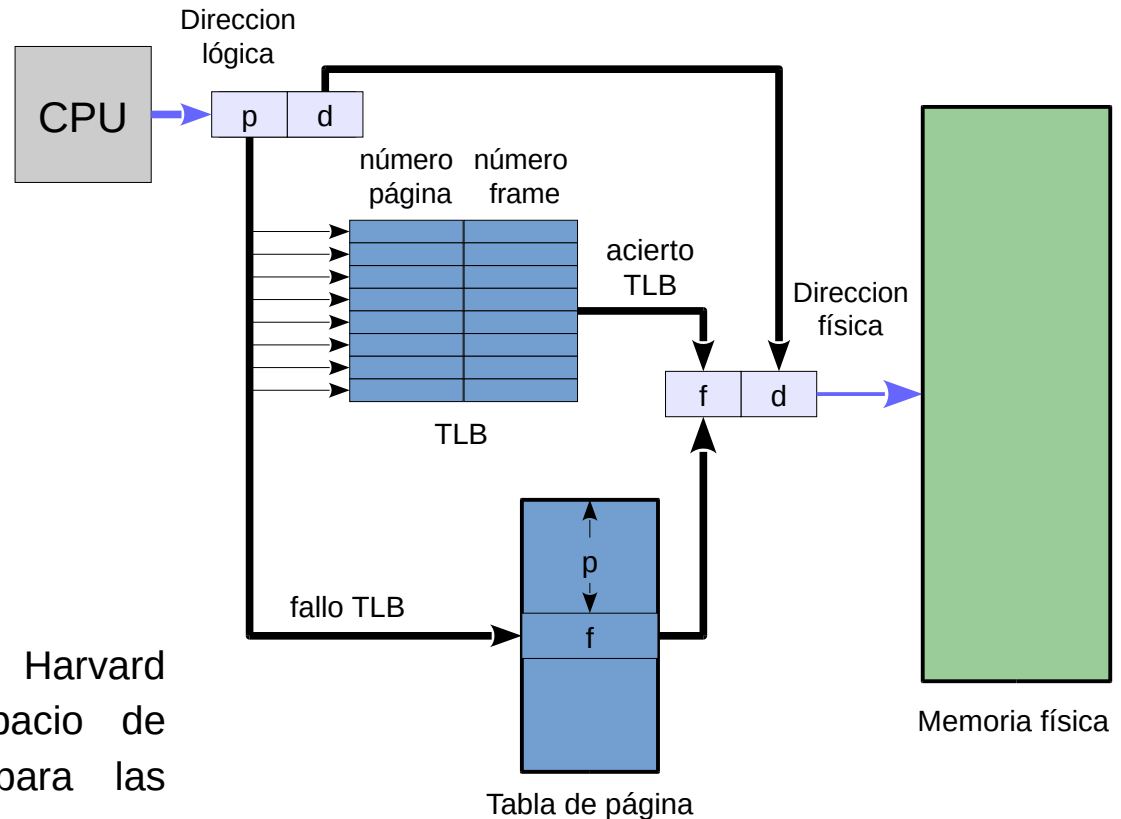
La TLB almacena las páginas y frames (p, f) de las páginas utilizadas con mayor frecuencia.

Sus principales característicos son su pequeño tamaño y elevada velocidad de traducción.

En una arquitectura de Harvard o Harvard modificada, puede existir un espacio de direcciones virtuales separados para las instrucciones y los datos.

Esto lleva al uso de distintos TLB para cada tipo de acceso, un búfer de instrucciones (ITLB) y un búfer de datos (DTLB).

Además, los TLB pueden tener múltiples niveles, un pequeño y totalmente asociativo que es extremadamente rápido (L1-TLB) y una más grande que es más lenta (L2-TLB).

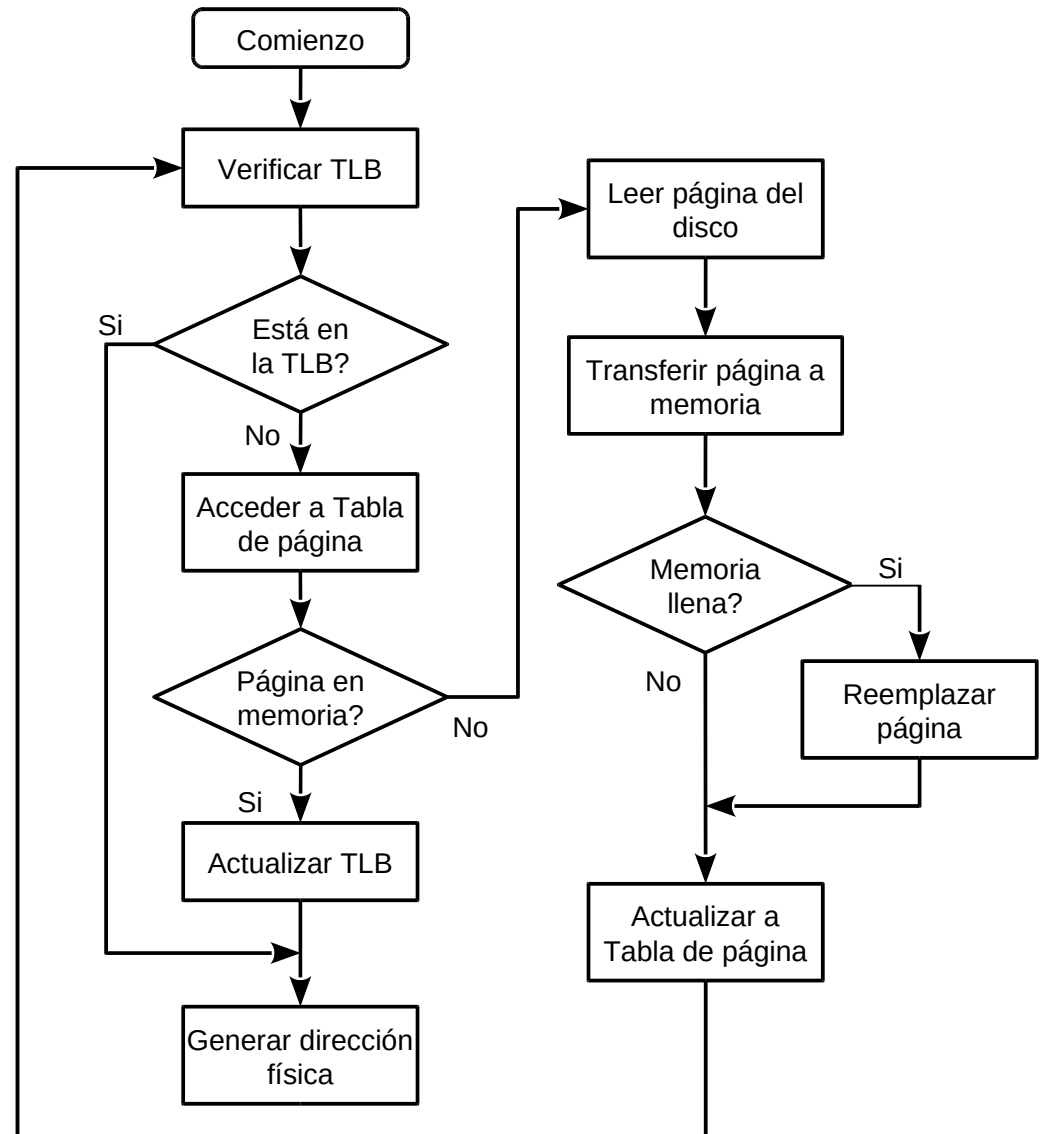


Paginación

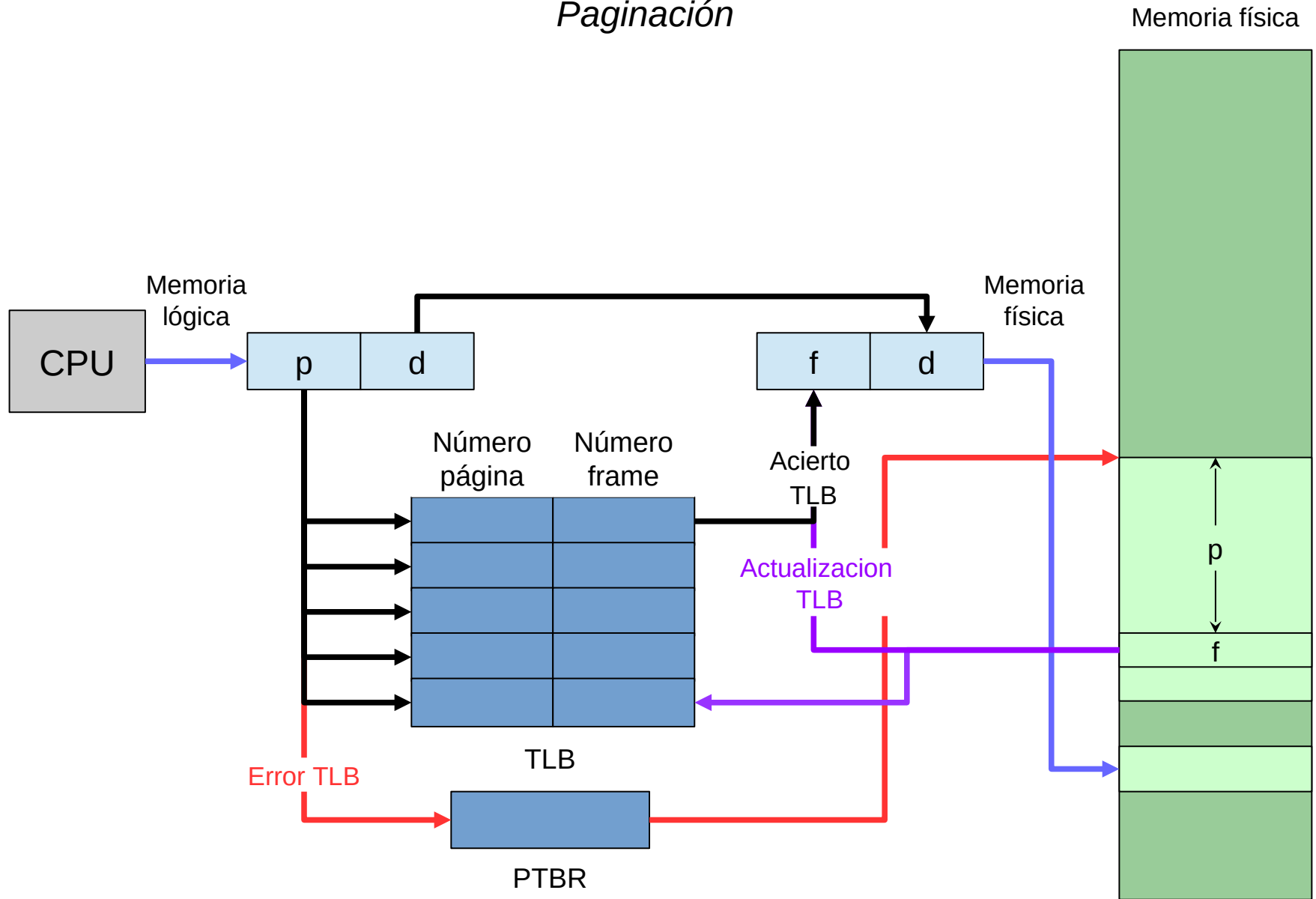
Si se produce un fallo de TLB, entonces la CPU verifica la tabla de páginas para encontrar la entrada de la tabla de páginas correspondiente.

Si el bit actual es 1, entonces la página está en la memoria principal y el procesador puede recuperar el frame para formar la dirección física. Finalmente, el procesador actualiza el TLB incluyendo la nueva entrada.

Si el bit actual es 0, entonces la página deseada no está en la memoria principal y se genera una excepción de error de página, la cual ejecuta la rutina de manejo de fallo de página.



Paginación

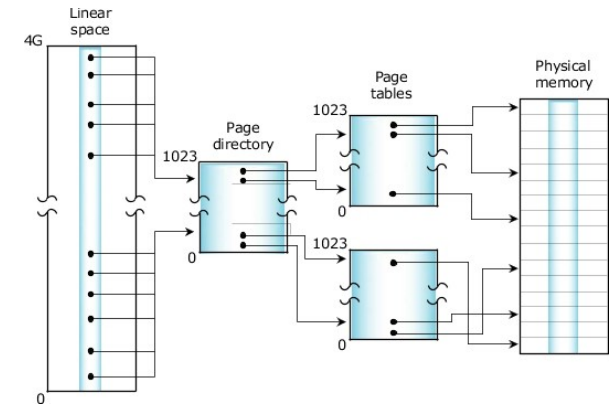
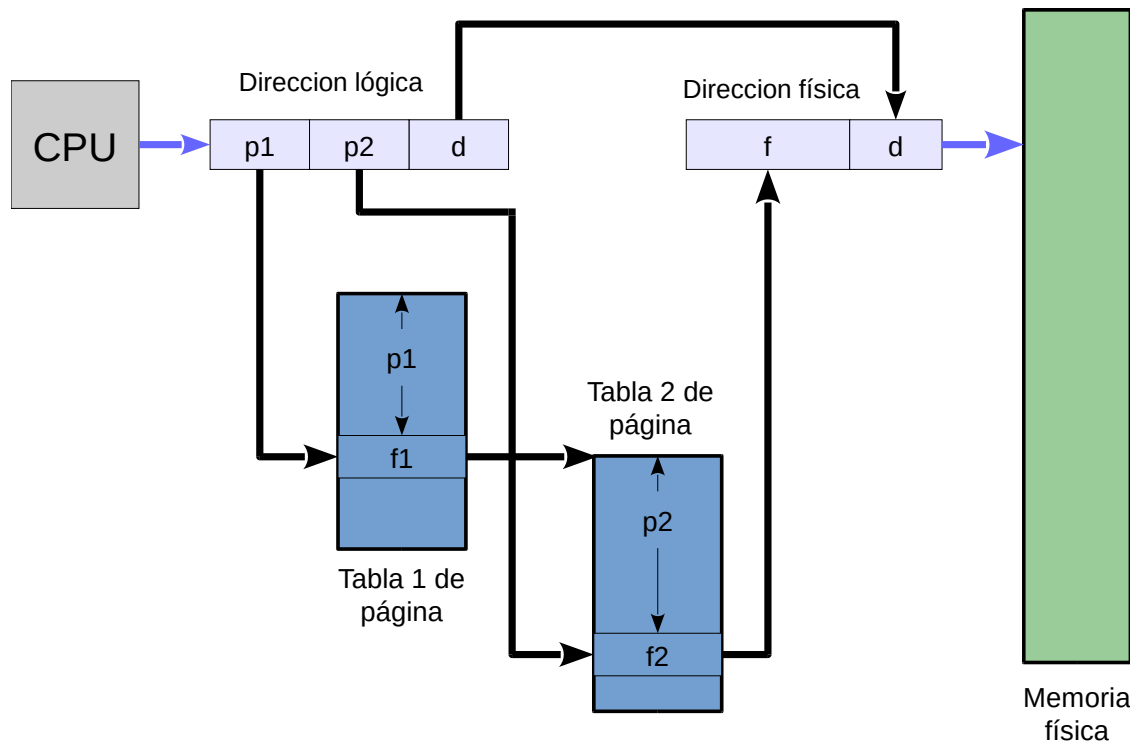


Paginación

El problema con las tablas de páginas es que pueden ser muy grandes.

Por ejemplo: *El espacio de direcciones de 32 bits con páginas de 4 KB implica que la tabla de páginas de un proceso tiene 1 millón de entradas.*

Es difícil encontrar una asignación contigua de al menos 1 MB para cada tabla de páginas.



Una solución a este problema consiste en subdividir las tablas para que puedan alojarse en la memoria al permitir que se disemine en partes no contiguas generando esquemas de paginación de múltiples (N) niveles.

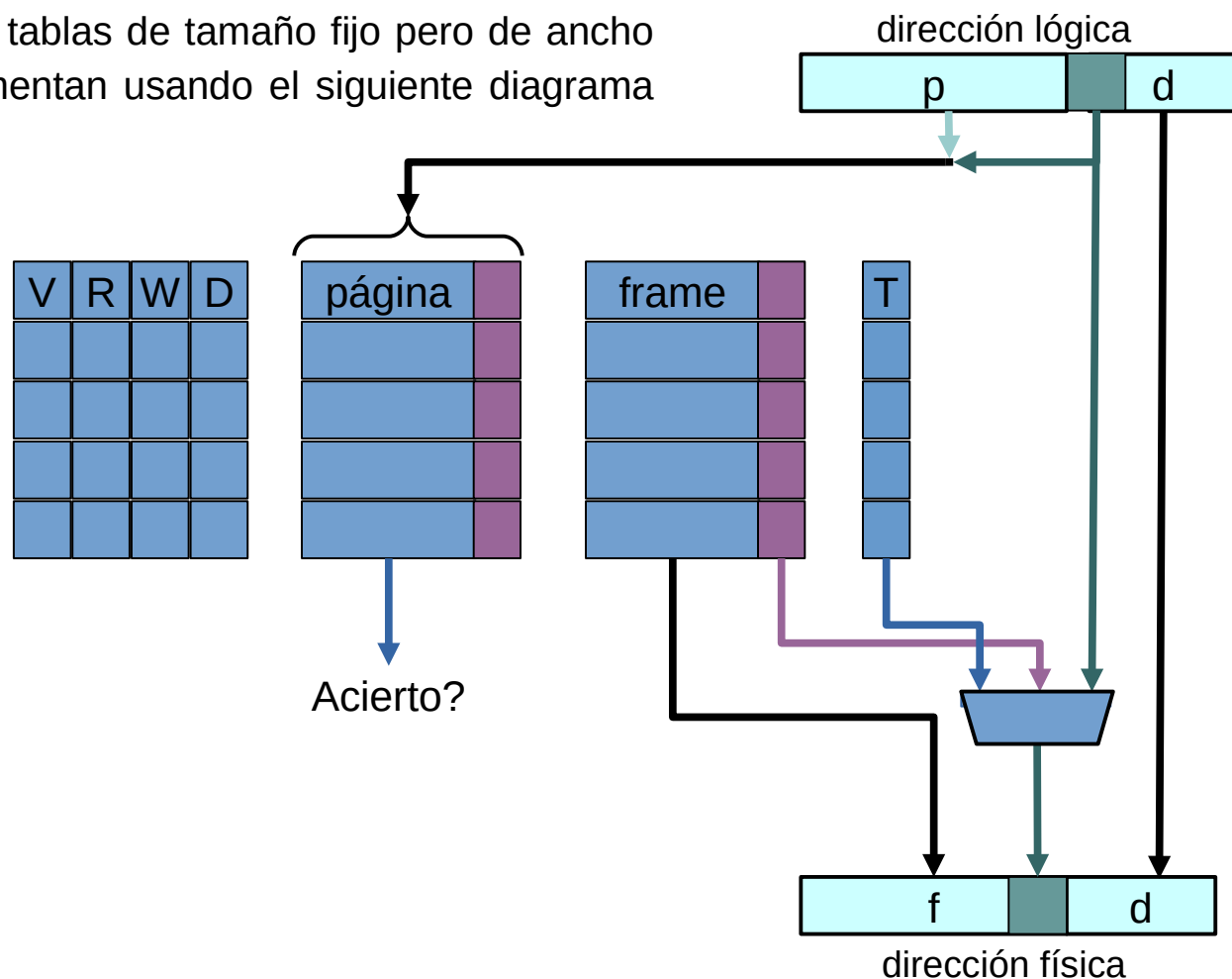
Manejo de memoria

Unidad de Manejo de Memoria

Otra solución consiste en el uso de páginas y frames de tamaño variable. Esta idea conduce a tablas de tamaño fijo pero de ancho variable, las cuales se implementan usando el siguiente diagrama de bloques.

En este esquema, los bits más bajos de la dirección de la página *p* también se pueden utilizar como parte del desplazamiento *d*.
diagrama de bloques.

Para resolver este problema se introduce un multiplexor, para seleccionar la fuente de información correcta, y bits extras (*T*) para indicar el tamaño de la página.



Paginación

La protección de memoria se logra asociando los bits de protección correspondientes con cada frame.

El bit de proteccion se utiliza para indicar si sólo se puede leer; si se puede escribir; si se puede ejecutar; entre otros.

Normalmente, estos bits se almacenan en la tabla de página.

Estos bits se agregan a cada entrada de la tabla de páginas:

- **valid** indica que la pagina asociada es válida en el espacio de direcciones lógicas del proceso y por lo tanto puede ser utilizada;
- **invalid** indica que la página asociada no pertenece al espacio de las direcciones logicas del proceso.

Si el sistema tiene una **tabla de registros de longitud de páginas** puede verificar si la direccion es válida.

Cualquier error puede ser atendido a traves de una excepcion del procesador.

page 0
page 1
page 2
page 3
page 4
page 5

frame number		valid-invalid bit
0	2	v
1	3	v
2	4	v
3	7	v
4	8	v
5	9	v
6	0	i
7	0	i

page table

0	
1	
2	page 0
3	page 1
4	page 2
5	
6	
7	page 3
8	page 4
9	page 5
	⋮
	page n

Segmentación

La segmentación es una técnica de gestión de memoria que soporta la organización de la memoria desde la perspectiva del programador: una colección de unidades lógicas que contienen diferentes tipos de información sin un orden preestablecido.

El espacio de direcciones lógicas es un conjunto de segmentos, cada uno de los cuales tiene un nombre y una longitud.

Para facilitar la implementación, los segmentos están numerados.

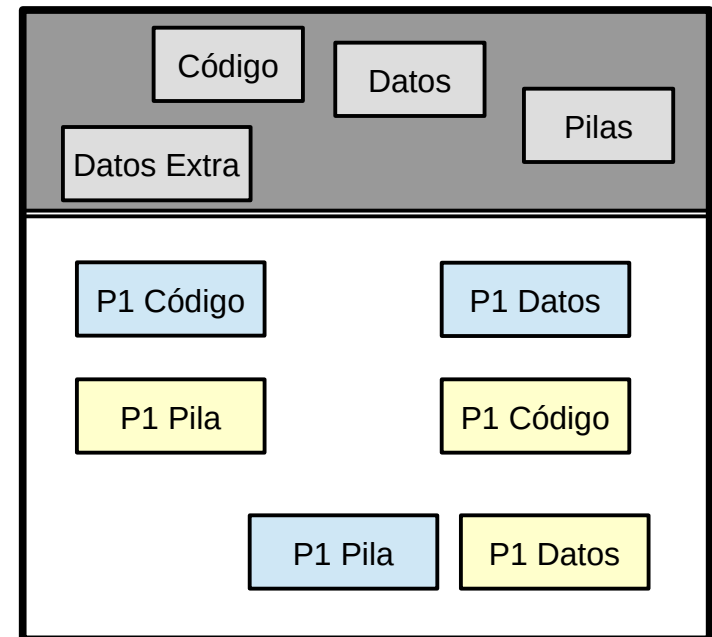
Una dirección lógica contiene dos tuplas:

`<segment-number, offset>`

Las direcciones identifican tanto el nombre del segmento como el desplazamiento dentro del segmento.

Cuando se compila un programa, el compilador crea automáticamente segmentos que reflejan el programa de entrada. Un compilador de C podría crear los segmentos de código; variables globales; heap y las pilas utilizadas por cada hilo y las biblioteca.

Memoria física



Segmentación

Los registros que definen al segmento son accesibles al programador cuando solo el procesador opera en modo supervisor o a traves de excepciones.

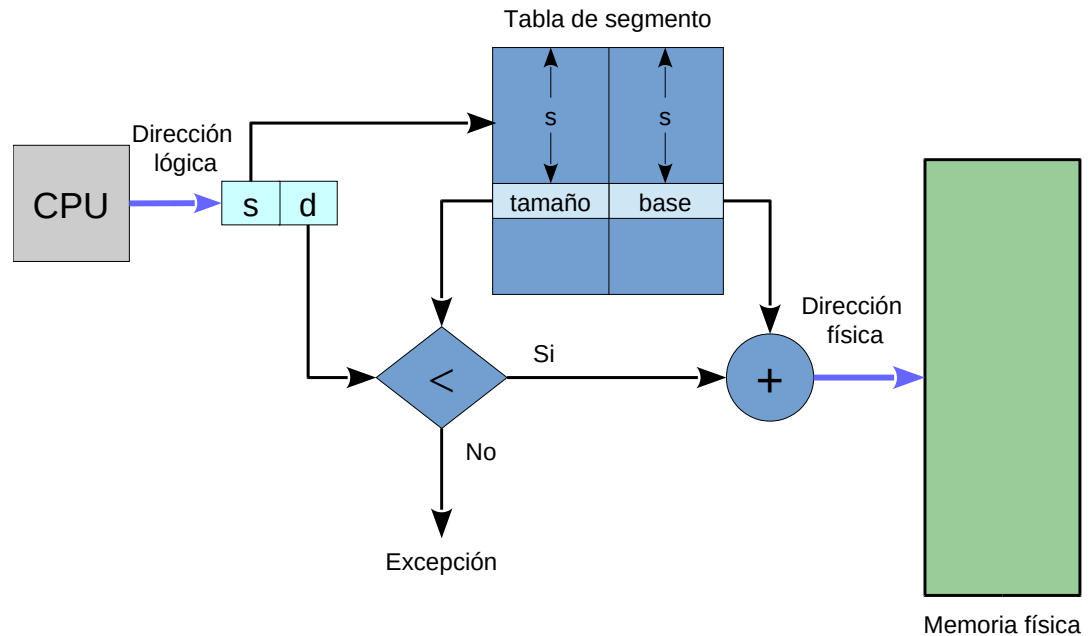
Estos registros estan almacenados en la tabla de segmento, la cual contiene dos datos:

- La direccion física de inicio del segmento (*base*); y
- El tamaño del segmento (*limite*)

Cada tabla de segmento esta definida a traves de

La **tabla de registros de base de segmento (STBR)**, la cual apunta la ubicación en memoria de la tabla de segmentos; y

La **tabla de registros de tamaño de segmento (STLR)**, la cual indica el número de segmentos utilizados por un programa.

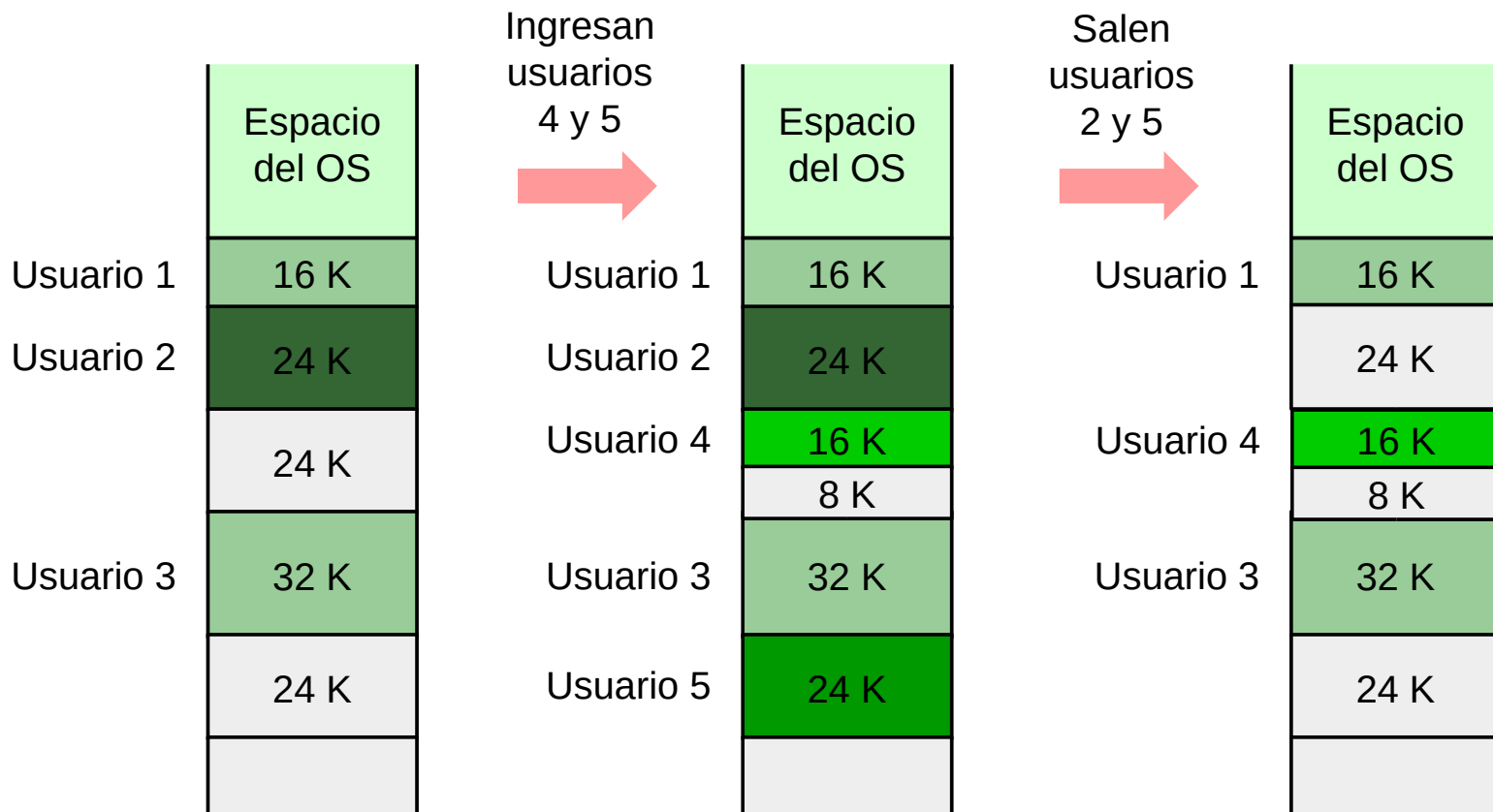


Manejo de memoria

Segmentación

Las ventajas de la segmentación es que permite que el espacio de las direcciones físicas de un proceso no sea contiguo.

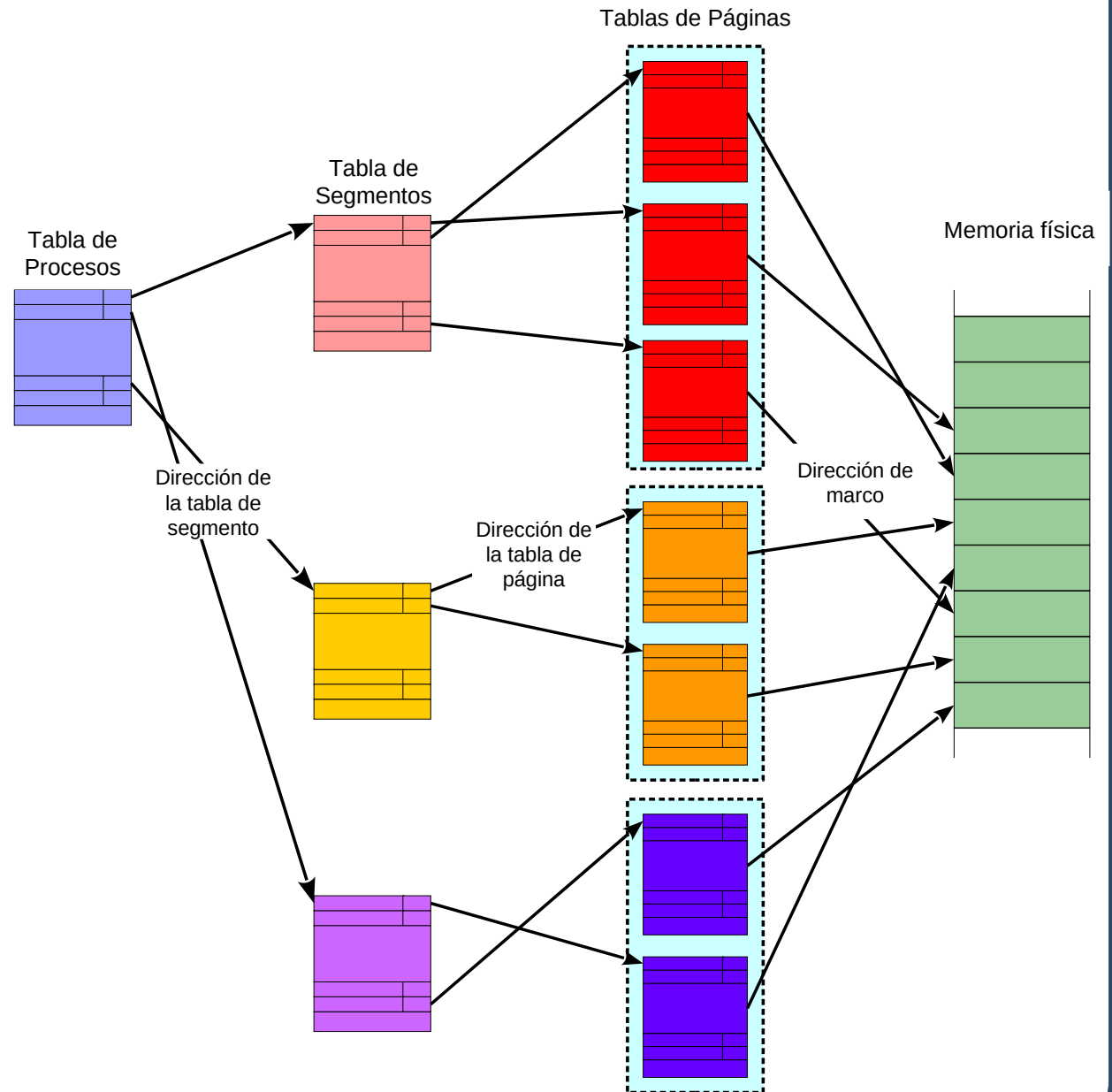
Las desventajas de la segmentación es que sufre de **fragmentación** y necesita ser compactado.



Manejo de memoria

Segmentación

La estructura de tablas de un sistema de paginación y segmentación.



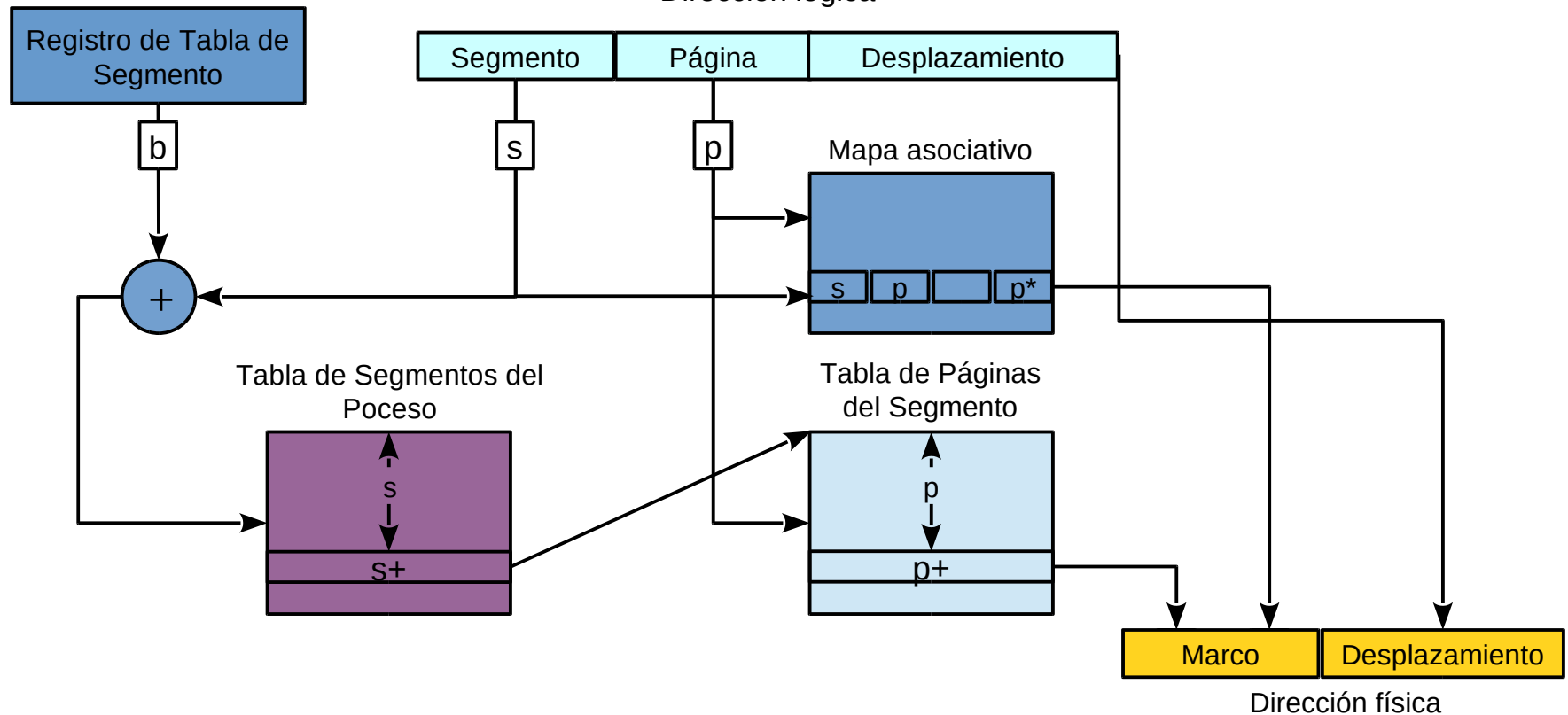
Manejo de memoria

Segmentación

Traducción de las direcciones lógicas con combinación de transformación asociativa/directa dentro de un sistema de segmentación y paginación

Dirección de la Tabla de Segmentos

Dirección lógica



Manejo de memoria

Virtualizacion

La memoria virtual es una técnica de gestión de memoria que proporciona una "*abstracción idealizada de los recursos de almacenamiento que están realmente disponibles en una máquina*" que "*crea la ilusión de una gran cantidad de memoria.*"

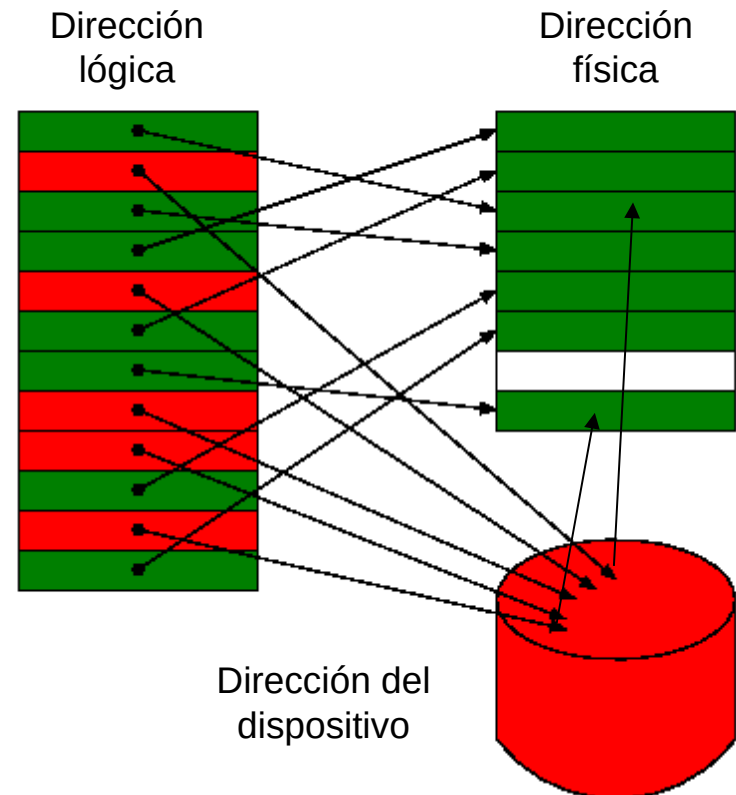
Utiliza una combinación de hardware y software para asignar las direcciones lógicas utilizadas por un programa a direcciones físicas en la memoria de la computadora.

La memoria aparece, para los procesos, como un espacio de direcciones contiguas o una colección de segmentos contiguos.

El sistema administra los espacios de direcciones virtuales y la asignación de memoria real a la memoria virtual.

La MMU traduce automáticamente las direcciones lógicas a direcciones físicas.

El sistema puede ampliar estas capacidades para proporcionar un espacio de dirección virtual que puede exceder la capacidad de la memoria real.



Manejo de memoria

Virtualizacion

Un proceso puede ser **intercambiado (swapped)** temporalmente entre la memoria y el medio de almacenamiento para su ejecución de acuerdo con los recursos disponibles.

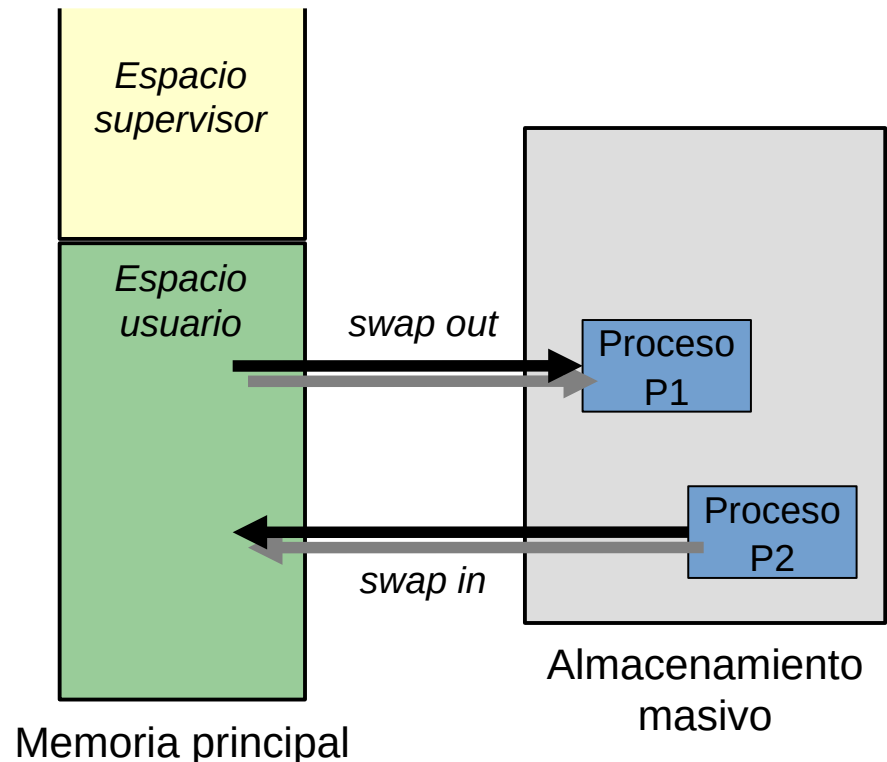
El sistema mantiene una **cola** de los procesos que están siendo ejecutados en la memoria que tienen una imagen en el medio de almacenamiento.

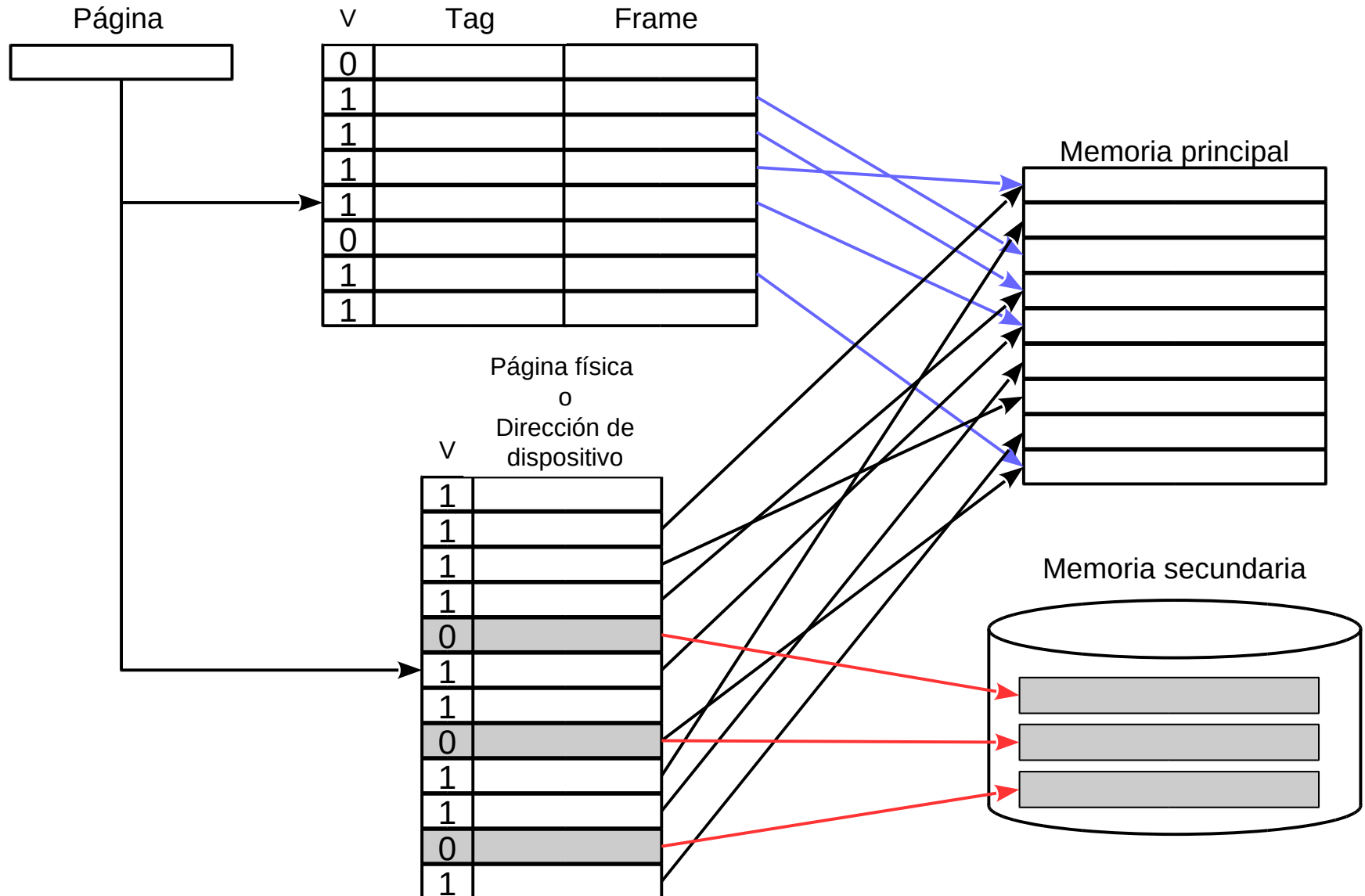
Esto permite que:

- El espacio de memoria utilizado por un proceso exceda la memoria física; y
- Aumenta el número de programas que se pueden utilizar simultáneamente.

Sin embargo, tiene como **cuello de botella** la velocidad del almacenamiento secundario, por lo que requiere de

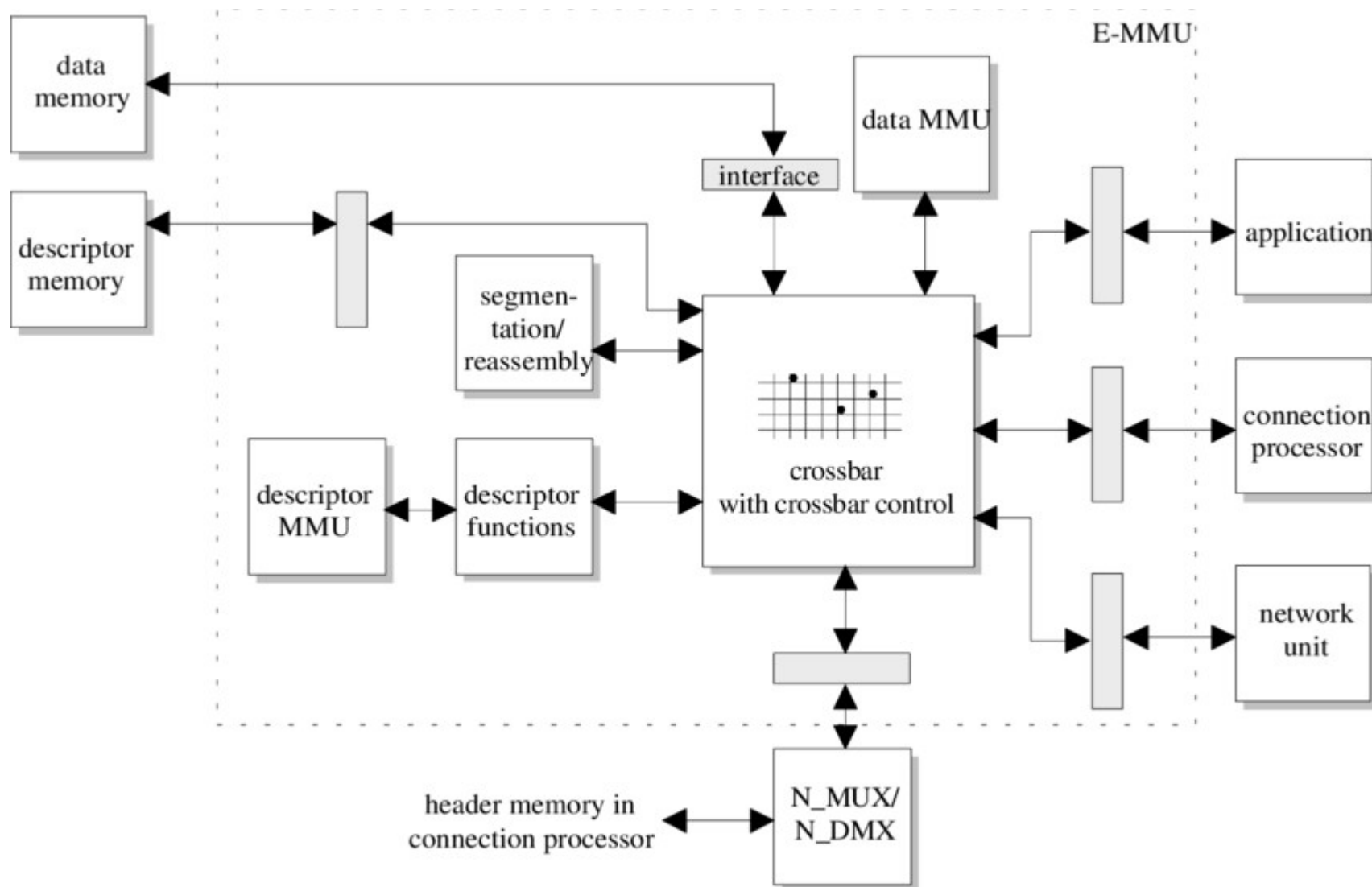
- Medios de almacenamiento veloces;
- Tamaño adecuado para almacenar toda la información y programas; y
- Permitir acceso a las copias de la imagen de memoria.



Virtualizacion

Manejo de memoria

Virtualización



Manejo de memoria

Virtualizacion - Usos

La virtualización de la memoria ha sido utilizada en:

- **Multiprogramación por lotes**

- Varios programas compartiendo un procesador mientras esperan por operaciones de E/S o llegada de nuevos trabajos;
- Base y limitación - traducción y protección entre programas (intercambio de programas completos pero no de memoria virtual paginada bajo demanda).

- **Tiempo compartido**

- Programas interactivos y eficientes;
- Traducción y protección de páginas de tamaño fijo sin fragmentación externa;
- Permitir que más trabajos compartan recursos limitados.

- **Monitores de máquinas virtuales**

- Ejecutar múltiples sistemas operativos en una máquina (original en los mainframes de IBM);
- Soporte de hardware para traducción/protección: OS invitado virtual -> OS invitado físico -> Hardware.

- **Necesidad de memoria virtual paginada**

- Asegurar la portabilidad entre máquinas con diferentes tamaños de memoria;
- Proveer protección entre múltiples usuarios o múltiples tareas;
- Compartir memoria física pequeña entre tareas activas;
- Simplificar la implementación de algunas funciones del sistema operativo.

Unidad de manejo de memoria

Manejo de memoria

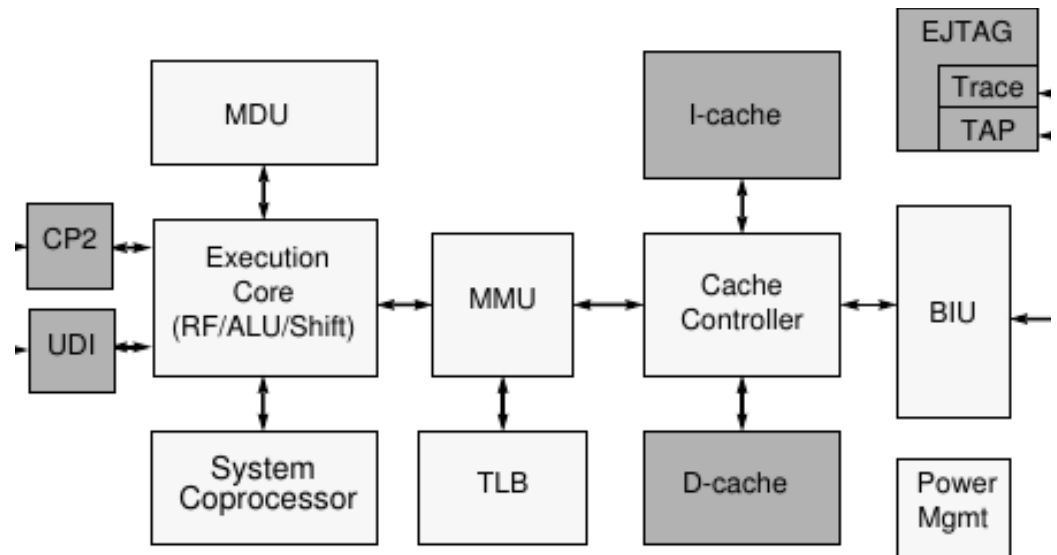
Unidad de Manejo de Memoria

La unidad de gestión de memoria es un dispositivo de hardware responsable del manejo de los accesos a la memoria por parte del procesador.

Las funciones de este dispositivo son la traducción de las direcciones lógicas (o virtuales) a direcciones físicas (o reales), la protección de la memoria, el control de caché y, en arquitecturas de computadoras más simples la conmutación de bancos de memoria.

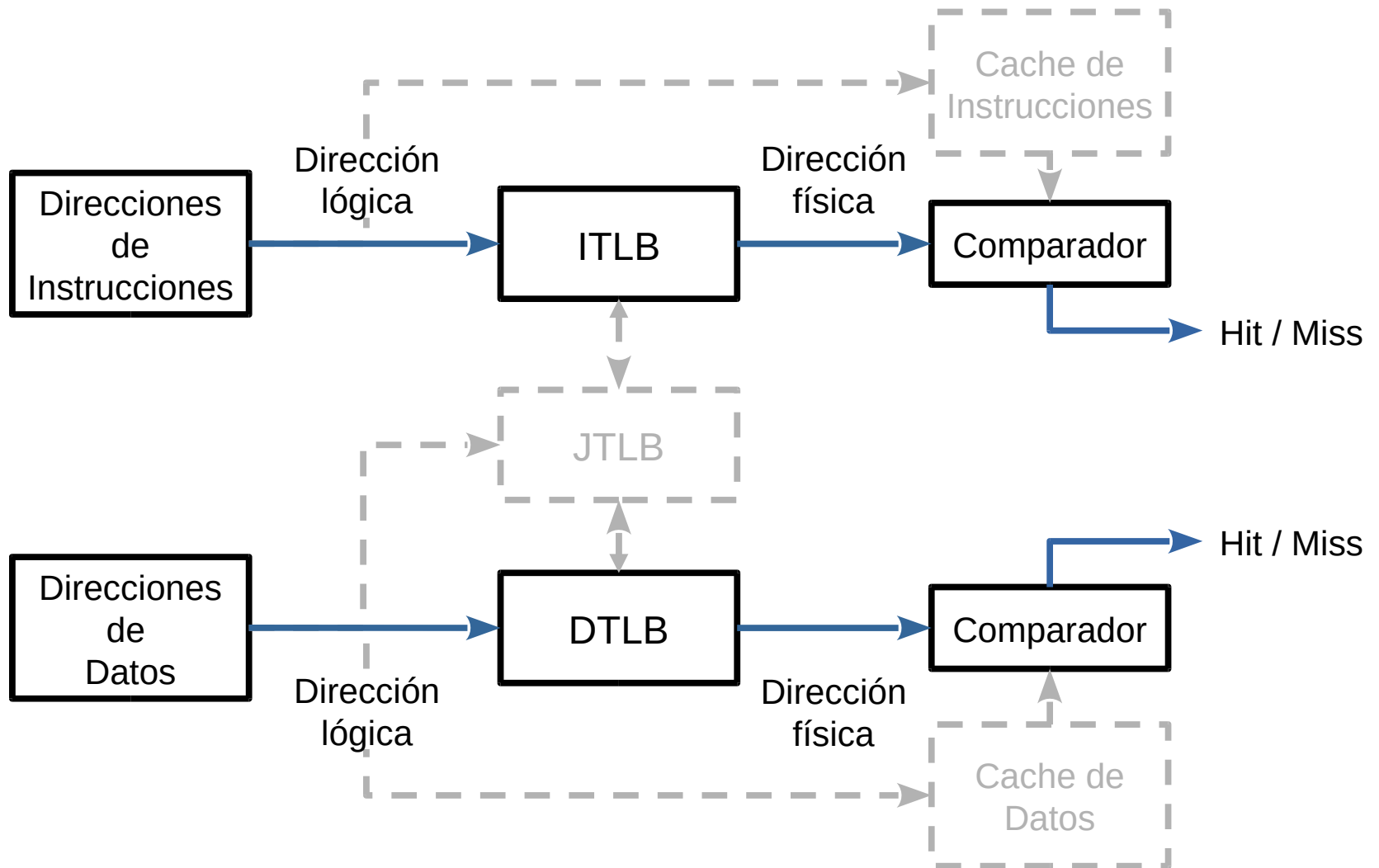
Un error de página puede indicar un error de software, que se puede prevenir usando la protección de la memoria como uno de sus beneficios: un sistema operativo puede usar la MMU para impedir el acceso no deseado a la memoria por un programa.

Una MMU también mitiga el problema de la fragmentación. Con la memoria virtual, un rango contiguo de direcciones virtuales puede asignarse a bloques no contiguos de memoria física.



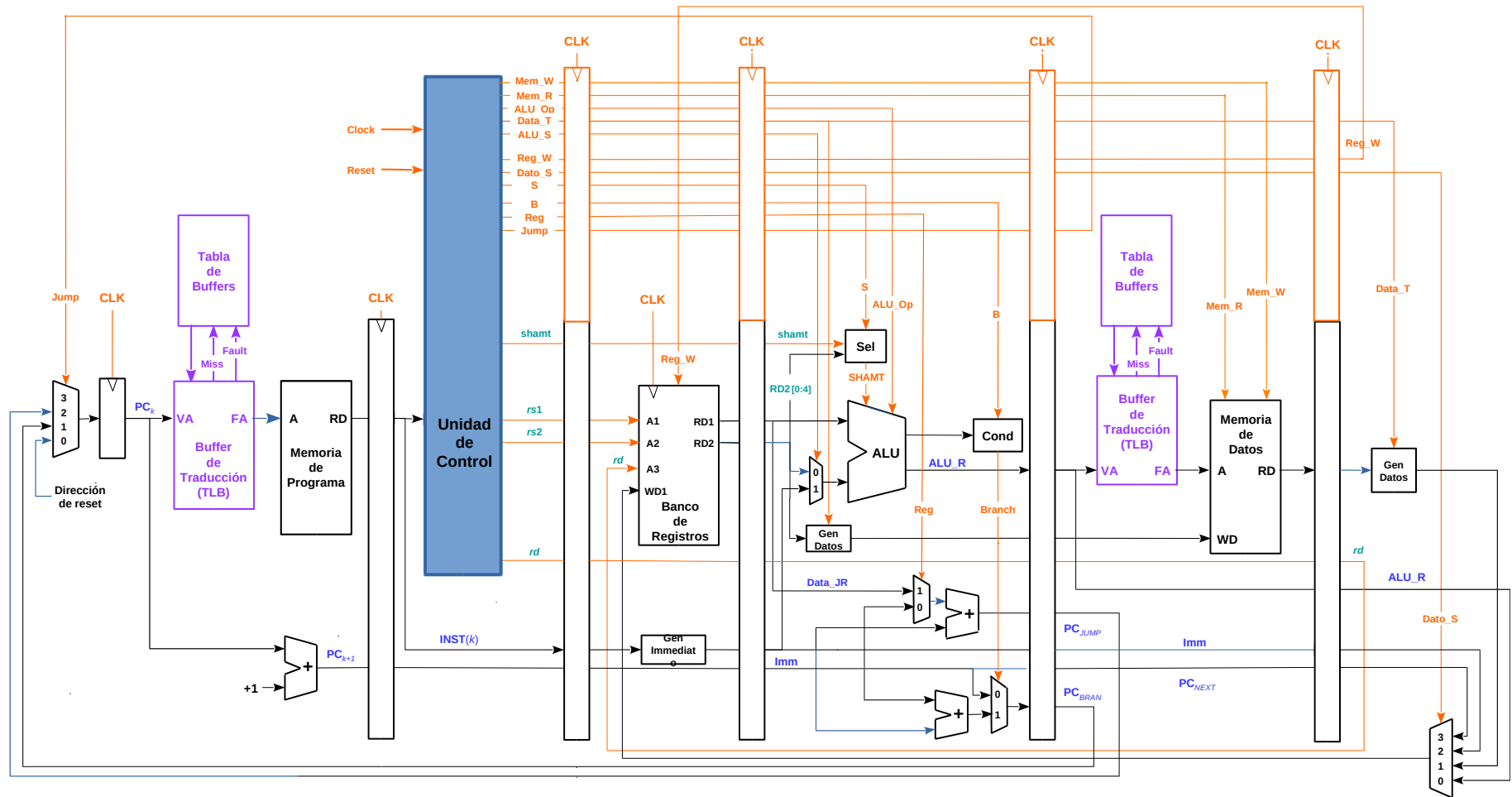
Manejo de memoria

Unidad de Manejo de Memoria



Manejo de memoria

Unidad de Manejo de Memoria



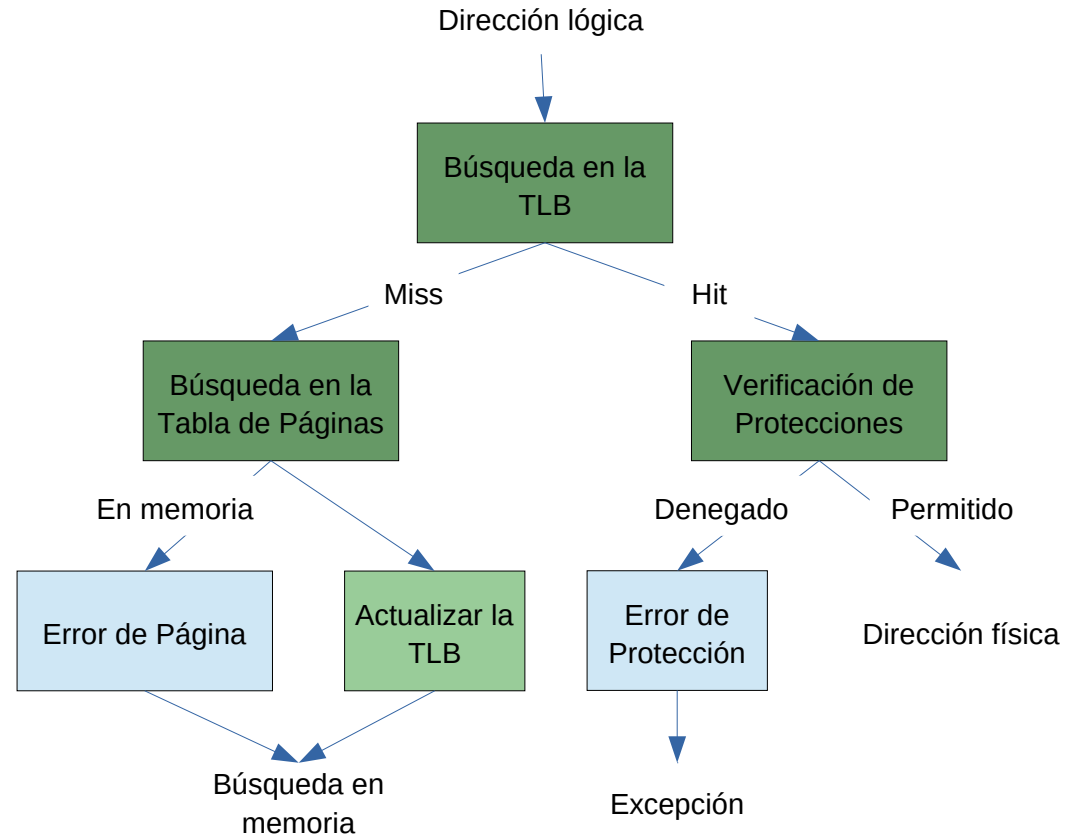
Manejo de memoria

Unidad de Manejo de Memoria

Los mecanismos para resolver errores en la operación de la TLB son

Software (MIPS, Alpha) - El error de direccion debido a que no se encuentra la dirección buscada en la TLB genera una **excepción** y el sistema operativo carga las nuevas TLB.

Hardware (SPARC v8, x86, PowerPC) - La unidad de manejo de memoria carga automáticamente las nuevas TLB. Si la página buscada es encontrada durante el proceso de recarga, la MMU genera una excepción para la instrucción original

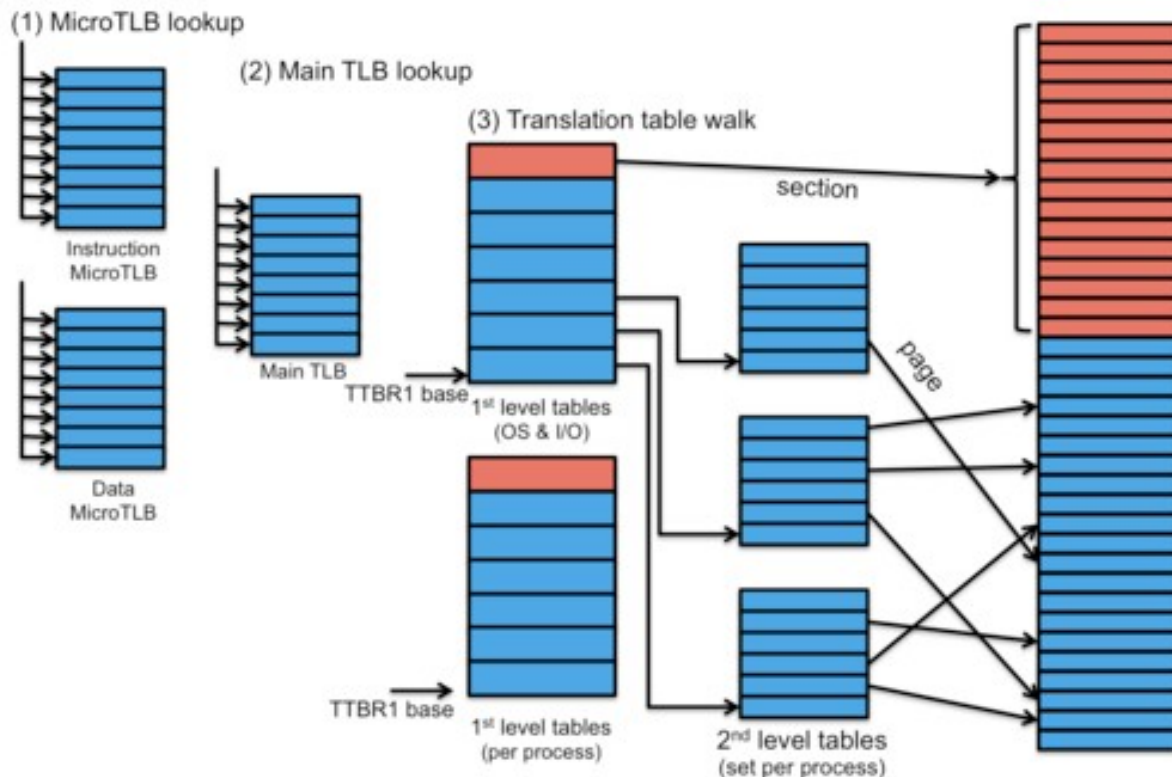


Manejo de memoria

Unidad de Manejo de Memoria - ARM

La MMU de la arquitectura ARM admite una estructura de tabla de páginas dos niveles, que depende de su tamaño **supersecciones** (bloques de 16 MB); **secciones** (bloques de memoria de 1 Mb); **páginas grandes** (bloques de 64 KB) y **páginas pequeñas** (bloques de 4 KB).

Una entrada en la tabla de primer nivel contiene un puntero a las tablas de segundo nivel o una dirección base de una sección o una supersección. Por lo tanto, si se utilizan páginas muy grandes, no tenemos que pasar por dos niveles de jerarquía.



La ventaja de las secciones y las supersecciones es que puede tener una gran región de memoria, como el sistema operativo, asignada con una sola entrada en la

La MMU se puede configurar para usar páginas pequeñas o grandes, así como secciones o supersecciones. Las secciones (o supersecciones) se pueden mezclar con las páginas

Manejo de memoria

Unidad de Manejo de Memoria - ARM

Cada acceso a la memoria se verifica con los permisos almacenados dentro tabla de páginas de cada bloque de memoria. Además, la entrada de la tabla de páginas también puede especificar cómo se comporta una región de la memoria con respecto al almacenamiento interno en caché, de la visibilidad y modificaciones a otros núcleos.

Las regiones de memoria pueden tener los siguientes atributos:

- **Execute never** – no permite las instrucciones tengan acceso a esta región de la memoria;
- **Read-only, read/write, no access** – estos modos configuran tanto el modo de usuario como el modo privilegiado. Por ejemplo, la memoria del kernel se puede etiquetar como sin acceso para el modo de usuario pero de lectura / escritura para la ejecución del modo de kernel.
- **Non-secure** – identifica la memoria como parte de una región confiable;
- **Sharable** – identifica si una región de la memoria se comparte con otros procesadores o se asigna a dispositivos. Varios modos están disponibles:
 - **Strongly ordered** – los accesos de memoria deben ocurrir en el orden del programa;
 - **Device/shared o device/non-shared** – a memoria se asigna a un dispositivo y el dispositivo se comparte o no con otros procesadores en el bus; y
 - **Normal/shared o normal/non-shared** – memoria regular que es compartida o no con otros procesadores en el bus.

Si el permiso no es válido para el acceso, la MMU atrapa con una señal de cancelación de memoria al procesador.

Unidad de Manejo de Memoria Intel

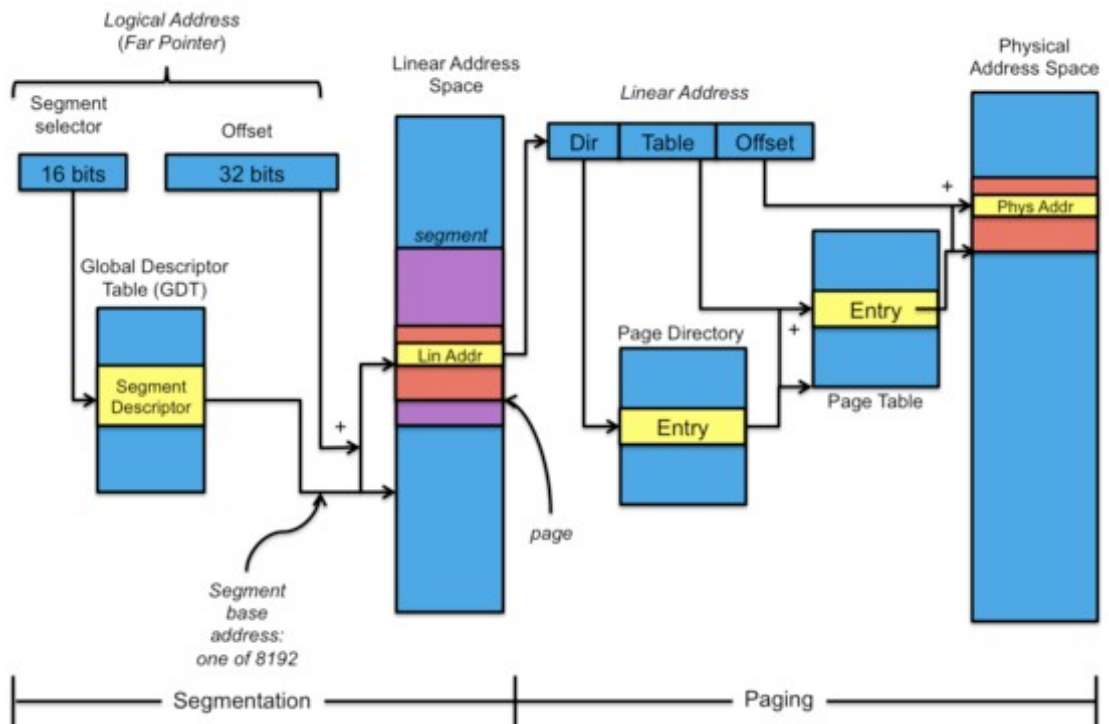
La arquitectura IA-32 admite un modelo combinado de segmentación y paginación. Hay dos tablas de segmentos que contienen las direcciones base de los segmentos:

- La tabla de descriptores locales (LDT) contiene información de segmento y es privada; y
- La tabla de descriptor global (GDT) contiene información de segmentos que se comparte entre todos los procesos.

El selector de segmento es un índice en estas tablas. Para seleccionar un segmento, el selector se carga en un registro de segmento. Además, cada instrucción que hace referencia a la memoria tiene un registro de segmento implícito, que puede ser anulado agregando un prefijo de segmento antes de la instrucción de la máquina.

Este modelo combinado utiliza la segmentación para generar una dirección lineal, la cual se trata como una dirección virtual y se traduce mediante la lógica de paginación de la MMU a través de una jerarquía de tablas de páginas de dos niveles.

La lógica de paginación se puede desactivar si no es necesaria.



Manejo de memoria

Unidad de Manejo de Memoria - Intel

Cada entrada de la GDT contiene la dirección base del segmento y banderas que definen los atributos del segmento

- **S** – esta bandera determina si el segmento contiene código o datos;
- **Accessed** – esta bandera determina si el segmento ha sido accedido desde la última vez que el sistema operativo lo reseteo;
- **Dirty** – esta bandera determina si el segmento ha sido modificado desde la última vez que el sistema operativo lo reseteo;
- **Data/write-enable** – esta bandera determina si el segmento es de solo-lectura o lectura-escritura;
- **Data/expansion direction** – esta bandera determina el lugar donde se memoria al segmento cuando se incrementa su tamaño al cambiar el límite;
- **Code/execute-only or execute/read** – esta bandera determina si la información contenida en un segmento de código puede ser accedida como datos o sólo ejecutada;
- **Conforming** – esta bandera indica si el código contenido en el segmento puede continuar su ejecución cuando el nivel de privilegio se eleva.